



One-pass online learning under evolving feature data streams: A non-parametric model

Han Zhou ^{a,b}, Hongpeng Yin ^{b,*}, Bin Wang ^a, Chenglin Liao ^c

^a The Department of New Networks, Peng Cheng Laboratory, Shenzhen, 518055, China

^b The School of Automation, Chongqing University, Chongqing, 400044, China

^c National Elite Institute of Engineering, Northwestern Polytechnical University, Xi'an, 710072, China

ARTICLE INFO

Keywords:

Data streams
Dynamic feature space
Evolving features
Online learning
One-pass learning

ABSTRACT

In real-world applications, data streams naturally evolves and thus may exhibit a dynamic feature space, wherein new features appear and old ones disappear. Online learning under such circumstances necessitates simultaneous learning from increasing data volume and adaptation to the dynamic feature space in real time. While several methodologies have been proposed to tackle this challenge, many of them rely on strong assumptions regarding evolving interaction manners. Instead, this study adopts a broader perspective aimed at facilitating learning from arbitrarily evolving features without any strict assumptions in the previous work. Then, we present an online learning method based on a non-parametric kernel model. This model accommodates data streams with both continuous instances and evolving features through simple deduction and addition operations. Theoretical analysis shows the sublinear regret $\mathcal{O}(\sqrt{T})$ of the proposed method. Empirical studies show the capability to adapt not only to the previously constrained evolving features but also to the more arbitrarily evolving features.

1. Introduction

In many of real-world applications, one may face the problem that data usually come continuously in the form of data streams. Consequently, online learning has emerged as an efficient solution for managing such scenarios. In general, the principal objective of online learning is to incrementally construct predictive models on a sequentially incoming stream of samples. Various methods of online learning have been actively proposed in literature. Representative techniques include the online convex optimization [1], ensemble learning [2] and many others [3].

Despite extensive research in the field of online learning, most algorithms operate under the assumption of fixed features. Whereas, this assumption is not always true, because in some applications, features can evolve in such a way that the new one appears and the old one disappears without any predefined interactions [4,5]. This leads to “Dynamic Feature Space”. One such example can be found in the context of online customer behavior analysis. Consider a scenario where a company introduces a new product. This introduces new features, such as product categories, purchase frequencies, or payment methods. In this case, online recommendation algorithms must incorporate these new features to accurately predict customer preferences. On the other

hand, if a company discontinues certain products, the associated features become obsolete. Online algorithms must then dynamically adapt by removing these features to avoid redundancy and reduce computational load. We call the data streams characterized by the potential for features to appear and disappear “Evolving Feature Data Streams”. The ability to handle the appearance and disappearance of features in real-time is crucial for maintaining up-to-date, efficient systems.

The key challenge to deal with evolving feature data streams lies in the development of online learning models that can effectively learn from increasing training data while flexibly adapting to dynamic feature spaces. Recent investigations have ventured into this domain, albeit with solutions that remain partial in nature. As shown in Fig. 1(a), The work [6,7] explores a setting of incremental feature learning, allowing the arriving data instances to carry different sets of features, yet later instances are assumed to include monotonically more features than the earlier ones. Namely, they cannot deal with the feature disappearance. The research in [8,9] considers the situation where new features appear and old ones disappear. However, they necessitate an overlapping period during which both the new and old features coexist, as visualized in Fig. 1(b) and in Fig. 1(c). Another approach that accommodates both the feature appearance and disappearance can be found in [10]. Regrettably, this approach also relies on a synchronized

* Corresponding author.

E-mail address: yinhongpeng@gmail.com (H. Yin).

<https://doi.org/10.1016/j.patcog.2025.111719>

Received 24 December 2024; Received in revised form 29 March 2025; Accepted 16 April 2025

Available online 24 May 2025

0031-3203/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

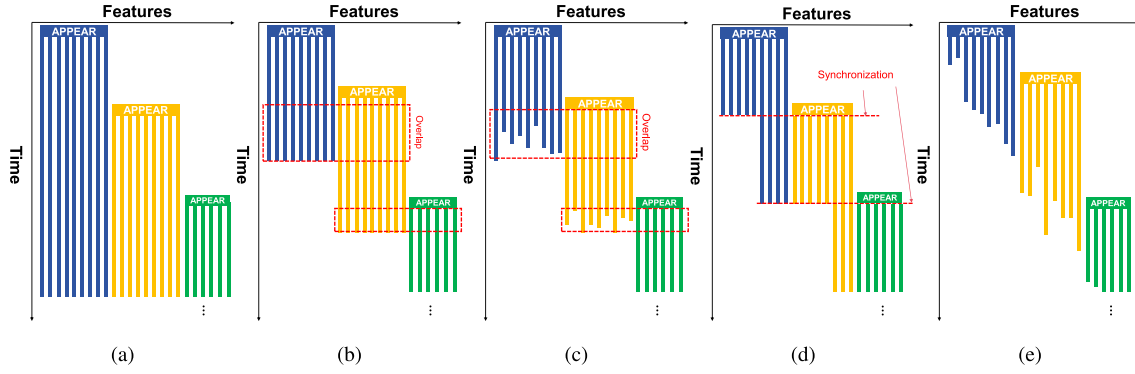


Fig. 1. Different feature evolution settings in related work. (a) Trapezoidal Evolution [6]: features continuously appear; (b) Overlap Evolution [8]: disappeared and appeared features must coexist in an overlap period; (c) Relaxed Overlap Evolution [9]: disappeared and appeared features must coexist in an incomplete overlap period; (d) Synchronized Evolution [10]: feature appearing and disappearing must occur at the same time; (e) General Evolution (Ours): arbitrary evolving without requirement of overlap or synchronization.

interaction assumption: the appearing and disappearing should occur at the same time, as visualized in Fig. 1(d). In summary, recent progress in learning from evolving features is usually marked by strict interaction assumptions, thereby limiting its applicability to more general and unpredictable evolving scenarios.

This study considers evolving feature data streams, with diminishing the assumptions held in the previous work: requiring interaction manner and prerequisite of feature dynamics (shown in Fig. 1(a-d)). In particular, in our general setting, as shown in Fig. 1(e), (1) features may appear or disappear arbitrarily, without interaction assumption; (2) features may evolve unpredictably, without prerequisite of when and which feature may appear or disappear. In this situation, this study introduces a novel method called Online Learning under Evolving Features based on Kernel Learning (OLEF-KL). It continuously reformulates the optimization problem by incorporating a flexible projection operation that adjusts the input dimensions, thus addressing evolving features in an online manner. Our contributions are listed below:

- This study introduces a general learning problem for evolving features, without imposing any assumptions or prerequisites related to feature evolution as seen in previous studies.
- This study proposes the OLEF-KL algorithm. It can continuously adapt not only to the previously strictly assumed evolving features but also to generally evolving features.
- This study guarantees the satisfying learning performance of OLEF-KL. The theoretical analysis under some mild assumption shows its upper performance bounds.

The rest of the article is organized as follows. Section 2 introduces related work. Section 3 formulates the problem of online learning under evolving feature data streams and proposes the OLEF-KL approach. The theoretical guarantee of OLEF-KL is given in Section 4. The experimental results are displayed in Section 5. Finally, we conclude this study in Section 6.

2. Related work

2.1. Online learning

The principal objective of online learning is to incrementally construct predictive models on a sequentially incoming stream of samples. Denote a sequence of data as $\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_T, y_T)\}$. As t represents the current time stamp, the learning objective is to predict the corresponding label $\hat{y}_t$ for a given sample $\mathbf{x}_t$. The prediction is made according to the previously learned model. Afterward, the true label y_t is revealed and the loss $\mathcal{L}(y_t, \hat{y}_t)$ determined.

Online learning can be easily achieved by stochastic optimization techniques such as online gradient descent [11,12] and passive aggressive algorithm [13,14]. Second order optimization methods can be

employed as well, such as online newton [15], confidence weighted learning [16]. Lazy learners such as k-nearest neighbor [17] lends itself to online scenarios by easily including a new training sample into the training set. Prototype-based models, such as online kernel learning [18,19], can also realize online learning schemes since they rely on the gradient technique. Online kernel learning can map data into high-dimensional reproducible kernel Hilbert spaces, facilitating efficient learning in transformation spaces. However, it faces a challenge known as the “kernelization curse”, resulting in the generation of an unlimited number of support vectors. One major challenge of the online kernel method is the computational efficiency and scalability due to the curse of kernelization. One possibility to dealing with this problem is setting a budget for the number of support vectors. Popular strategies include forgetting old support vectors [20], selecting representative support vectors [21] and embedding removed support vectors in the remaining ones [22].

Interestingly, these methods have already been accompanied by exact mathematical investigations. The performance of the learned model f_t can be measured by regret:

$$\text{Regret} = \sum_{t=1}^T \mathcal{L}(f_t; \mathbf{x}_t, y_t) - \sum_{t=1}^T \mathcal{L}(f^*; \mathbf{x}_t, y_t). \quad (1)$$

$\mathcal{L} : \mathbb{R} \rightarrow \mathbb{R}$ is a convex loss function which is Lipschitz continuous with Lipschitz constant L . The notation f^* is the best learner that minimizes the convex loss function. The concept of regret quantifies the difference between the cumulative loss accumulated by an online learning algorithm and the accumulative loss that would have been experienced had the algorithm known the best action in advance.

Most online learning approaches assume a fixed feature space. When this assumption is unfortunately violated — such as when features appear or disappear — issues can arise. The model may struggle to forget old features or miss important information if it cannot incorporate new ones. This highlights the need for methods that can effectively learn from an increasing amount of training data while flexibly adapting to evolving feature spaces.

2.2. Online learning under evolving features

Progress has been established in the literature, but strong assumptions often apply, impeding applicability for real-world systems. One of the early work [7] concerns online learning under trapezoidal evolving feature, as shown in Fig. 1(a). It introduces a trapezoidal data stream in which both samples and features continuously increase in a monotonic manner. The learning model particularly augments the weight vector once new features appear. Similar setting also can be found in [23–28]. We are aware of work [29–31] that does not use linear but non-linear model. They generally establish a relationship between the pre-and-post evolving features by a generative model, and try to reconstruct

all disappeared features from the new ones in the later procedure. This setting can also be viewed as the trapezoidal evolution.

The work in [8] is capable of dealing with the situation where new features appear and old ones disappear, as visualized in Fig. 1(b) and Fig. 1(c). A key technique shared by these methods is to establish a mapping relationship between two feature spaces. As such, once the old features fade away, their information can be reconstructed via the mapping, aiding the new learner join to make highly accurate predictions. Unfortunately, this technology necessitates a complete overlap period during the alternation between old and new feature spaces, so that they can capture the correlation between new features and old features through a projection matrix. Similar setting also can be found in [9,32–35].

Other approaches that accommodate both appearance and disappearance of the features can be found in [10,36,37]. They compress crucial information from disappeared features into a function of remaining features and learn alongside appeared features. Unfortunately, they rely on a strict interaction assumption: appearing and disappearing should occur at the same time, as shown in Fig. 1(d). Meanwhile, these approaches operate under the prerequisite that the disappeared features are known in advance, thus confining their applicability to unpredictable evolving scenarios.

The evolution setting in Fig. 1(e) is a more general paradigm that we introduce to facilitate learning from any data stream with dynamic features, without assumptions and prerequisites for feature interaction. Of particular relevance to our study is the work in [5,38–41]. Work [38] deals with evolving features by dynamically projecting the classifier and training samples onto their shared feature subspace. In particular, the projection weights were dynamically adjusted by learning projection confidence. Work [39,40] replaces the projection weights of work [38] by feature uncertainty and introduces further modifications to align itself with the challenges posed by class imbalance and concept drift. Work [41] further considered the sparsity in data streams by introducing the $\ell_{1,2}$ norm. Unfortunately, progress in this setting has been almost exclusively on finite-dimensional linear classification, failing to capture nonlinear patterns in dynamic feature space.

2.3. Brief summary

Naive online learning methods assume a static feature space. When features change, these methods can lead to potential issues, such as overfitting to outdated features or failing to incorporate relevant new information. Although significant progress has been made in accommodating feature evolution, many approaches still rely on strict assumptions about how features interact. In contrast, this paper relaxes these assumptions, allowing features to appear or disappear arbitrarily. Then, this study introduces the OLEF-KL algorithm, not only adapting to the previously strictly assumed evolving features but also to generally evolving features. Recently, we found that studies [42,43] address a similar setting, employing rough set theory to handle changing data volumes and features. In contrast, our method adopts a non-parametric approach designed for evolving feature data streams.

3. Methodology

3.1. Problem setting

We start with an introduction of notations and the problem setting. Let $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_T, y_T)\}$ be a sequence of labeled data with evolving features, where $\mathbf{x}_t \in \mathbb{R}^{\mathbf{d}_t}$, $y_t \in \{-1, +1\}$ and $\mathbb{T} = \{1, \dots, T\}$. The dimension of full feature space is denoted as $\mathbb{D} = \bigcup_{t=1}^T \mathbf{d}_t$. Our goal is to learn a classifier $f_t : \mathbb{R}^{\mathbf{d}_t} \rightarrow \mathbb{R}$ to predict the class assignment $\hat{y}_t$ for $\mathbf{x}_t$ by $f_t(\mathbf{x}_t)$. Ideally, at each round of online learning, once

received a labeled sample $(\mathbf{x}_t, y_t)$, we solve the following empirical risk minimization problem to update f_t :

$$f_t = \arg \min_f \left\{ \frac{1}{t} \sum_{\tau=1}^t \mathcal{L}(f; \mathbf{x}_\tau, y_\tau) \right\}, \quad (2)$$

where $\mathcal{L} : \mathbb{R} \rightarrow \mathbb{R}$ is a convex loss function which is Lipschitz continuous with Lipschitz constant L .

3.2. Proposed method

Recent progress in evolving feature data streams has been almost exclusively in finite-dimensional linear classification. Important exceptions to this limitation are given in [24,32]. However, it applies strong assumptions about feature evolution, which hinders applicability to real-world systems. Instead, this study employs kernelized methods and introduces a non-parametric model for online learning with evolving features.

Kernel methods operate by mapping data into Hilbert spaces and subsequently exploring linear relationships within these spaces. Let us rewrite the optimization problem in Eq. (2) to suit the situation containing evolving features at each trial t :

$$f_t = \arg \min_f \left\{ \frac{1}{t} \sum_{\tau=1}^t \mathcal{L}(f; \Pi_{\mathbf{d}_t} \mathbf{x}_\tau, y_\tau) + \Omega_{\mathcal{H}_K}(f) \right\}. \quad (3)$$

Here, $\mathcal{H}_K$ denotes the Hilbert space associated with the reproducing kernel function $\mathcal{K}(\cdot, \cdot) : \mathbb{R}^{\mathbf{d}} \times \mathbb{R}^{\mathbf{d}} \rightarrow \mathbb{R}$. We assume $\mathcal{K}(\mathbf{x}, \mathbf{x}) \leq 1$ for any $\mathbf{x}_t \in \mathbb{R}^{\mathbf{d}_t}$. The $\Omega_{\mathcal{H}_K}(f_t)$ is a regularization term. The projection operation $\Pi_{\mathbf{d}_t} \mathbf{x}_\tau : \mathbb{R}^{\mathbf{d}_\tau} \rightarrow \mathbb{R}^{\mathbf{d}_t}$ projects the sample $\mathbf{x}_\tau$ from dimension $\mathbf{d}_\tau$ to $\mathbf{d}_t$, whose elements from $\mathbb{R}^{\mathbf{d}_\tau}$ are the same as the original ones while others in $\mathbf{d}_t \setminus \mathbf{d}_\tau$ are imputed by zeros.

According to the representer theorem [44], the solution of Eq. (3) can be represented as a linear combination of the feature vectors corresponding to the training data points or support vectors:

$$f_t := \text{span}\{\mathcal{K}(\Pi_{\mathbf{d}_t} \mathbf{x}_\tau, \cdot) | \tau \in \mathbb{T} \setminus \{t\}\} \subseteq \mathcal{H}_K, \quad (4)$$

where $\mathbb{T}$ is the set $\{1, \dots, t\}$. This formulation also can be expressed as [44]:

$$f_t = \sum_{\tau=1}^{t-1} \alpha_\tau \mathcal{K}(\Pi_{\mathbf{d}_t} \mathbf{x}_\tau, \cdot), \quad (5)$$

where $\alpha_\tau = -\eta \nabla \mathcal{L}(f_t; \mathbf{x}_\tau, y_\tau)$ is the coefficient for the τ -th support vector $\mathbf{x}_\tau$, and $\eta > 0$ is the learning rate.

In order to quantify the similarity of feature space within different time, we further introduce the Jaccard metric:

$$J_{i,j} = \frac{|\mathbf{d}_i \cap \mathbf{d}_j|}{|\mathbf{d}_i \cup \mathbf{d}_j|}, \quad (6)$$

where $|\mathbf{d}_i \cap \mathbf{d}_j|$ represents the number of common features between feature set $\mathbf{d}_i$ and $\mathbf{d}_j$. $|\mathbf{d}_i \cup \mathbf{d}_j|$ represents the total number of unique features in sets $\mathbf{d}_i$ and $\mathbf{d}_j$. Clearly, we have $J_{i,j} \in [0, 1]$ and $J_{i,i} = 1$. A small Jaccard metric between samples indicates a substantial evolution in the set of features. Based on the Jaccard metric, we can further rewrite Eq (5) to the following form:

$$f_t = \sum_{i=1}^{t-1} \alpha_i J_{t,i} \mathcal{K}(\Pi_{\mathbf{d}_t} \mathbf{x}_i, \cdot), \quad (7)$$

To alleviate the computational complexity in solving problem Eq. (3) during the update of f_t , one can devise a lightweight updating rule tailored to better accommodate data streams. The fundamental concept revolves around solely incorporating the information from the most recent sample through a stochastic gradient descent iteration. When using the hinge loss and the model made an mistake prediction, the updating rule can be given by:

$$f_{t+1} = f_t - \eta J_{t,t} \nabla \mathcal{L}(f_t; \mathbf{x}_t, y_t) \mathcal{K}(\mathbf{x}_t, \cdot) \leftarrow \alpha_t = -\eta \nabla \mathcal{L}(f_t; \mathbf{x}_t, y_t) \quad (8)$$

$$\begin{aligned}
&= \sum_{\tau=1}^{t-1} \alpha_{\tau} J_{\tau,t} \mathcal{K}(\Pi_{\mathcal{d}_t} \mathbf{x}_{\tau}, \cdot) + \alpha_t J_{t,t} \mathcal{K}(\mathbf{x}_t, \cdot) \\
&= \sum_{\tau=1}^t \alpha_{\tau} J_{\tau,t} \mathcal{K}(\Pi_{\mathcal{d}_t} \mathbf{x}_{\tau}, \cdot).
\end{aligned}$$

As we can see, updating f_t only requires the addition of the newly observed sample and re-calculation of the Jaccard metric. Its additive operation is inherently designed to facilitate learning from continuously received samples.

3.3. Exponentially weighted decay

Our goal is to sequentially learn the function f_t , and subsequently predict classes $\hat{y}_t$ per time t from historical observations $\{(\mathbf{x}_{\tau}, y_{\tau}) | \tau \in \mathbb{T} \setminus \{t\}\}$, naturally placing more importance on recent samples. To this end, one possible adaptive counterpart to Eq. (7) is the exponentially-weighted decay updating:

$$f_{t+1} = \sum_{\tau=1}^t \rho^{t-\tau} \alpha_{\tau} J_{\tau,t} \mathcal{K}(\Pi_{\mathcal{d}_t} \mathbf{x}_{\tau}, \cdot). \quad (9)$$

The parameter $\rho \in (0, 1]$ is the so-termed forgetting factor. When $\rho < 1$, data in the distant past are exponentially down-weighted. Then, one can further truncate the support vectors with smallest coefficients to avoid the unbounded number of support vectors. The truncation strategy can be expressed by:

$$[\rho^{t-\tau} \alpha_{\tau}]_{\epsilon} = \begin{cases} \rho^{t-\tau} \alpha_{\tau} = 0, & \text{if } \rho^{t-\tau} \alpha_{\tau} < \epsilon; \\ \rho^{t-\tau} \alpha_{\tau} = \rho^{t-\tau} \alpha_{\tau}, & \text{others.} \end{cases} \quad (10)$$

where ϵ is a small constant and we set $\epsilon = 0.01$ in this study.

Both the high computational time and evolution adaptation motivate the introducing of the parameter ρ . (1) Computational Efficiency: As can be seen from Eq. (8), the number of calculations required for kernel evaluations grows rapidly with the number of support vectors. This brings huge burdens on computation and storage as time goes. A decay parameter enables our algorithm to forget older samples, which can lead to memory- and computation-efficient implementations. (2) Evolving Feature Adaptation: In the feature evolving scenarios, the importance of past observed samples may diminish as the changing of feature space. A time-decay parameter allows our algorithm to assign decreasing weights to older data, emphasizing more recent observations, which can lead to improved model adaptability to evolving feature spaces.

Remark 1. The time decay factor is also equivalent to the ridge regularization which penalizes the squared values of the coefficients, i.e. $\Omega_{\mathcal{H}_K}(f_t) = \|f_t\|_{\mathcal{H}_K}^2$. Let us consider the following optimization problem with ridge regularization in the Hilbert space:

$$\min_f \left\{ \mathcal{L}(f; \mathbf{x}_t, y_t) + \frac{\lambda}{2} \|f\|_{\mathcal{H}_K}^2 \right\}. \quad (11)$$

At each trial t , we update the classifier by gradient descent and it yields the following updating rule:

$$\begin{aligned}
f_{t+1} &= (1 - \eta\lambda) f_t - \eta \nabla \mathcal{L}(f_t; \mathbf{x}_t, y_t) \mathcal{K}(\mathbf{x}_t, \cdot) \\
&= \bar{\rho} f_t + \alpha_t \mathcal{K}(\mathbf{x}_t, \cdot) \\
&= \sum_{\tau=1}^t \bar{\rho}^{t-\tau} \alpha_{\tau} \mathcal{K}(\mathbf{x}_{\tau}, \cdot).
\end{aligned} \quad (12)$$

By adjusting the strength of the parameter ρ , we can control the degree of smoothness and stability in our model. This can prevent overemphasizing the importance of a support vector caused by assigning very large values to any single coefficient.

Algorithm 1 OLEF-KL: Online Learning under evolving feature data streams, based on kernel learning.

Require: Data stream $\mathcal{D}$, learning rate η_t , decay parameter ρ and a kernel function $\mathcal{K}(\cdot, \cdot)$.

```

1: for  $t = [1, T]$  do
2:   Receive a sample  $\mathbf{x}_t \in \mathcal{I}$ .
3:   Make a prediction using Eq. (13).
4:   Receive the truth  $y_t$  and suffer a loss  $\mathcal{L}(f_t; \mathbf{x}_t, y_t)$ .
5:   Calculate the Jaccard metric using Eq. (6).
6:   Update the prediction model  $f_t$  via Eq. (9) and Eq. (10).
```

Accordingly, a prediction for the next unlabeled samples can be made by:

$$f_{t+1}(\mathbf{x}_{t+1}) = \sum_{\tau=1}^t [\rho^{t-\tau} \alpha_{\tau}]_{\epsilon} J_{\tau,t} \mathcal{K}(\Pi_{\mathcal{d}_t} \mathbf{x}_{\tau}, \mathbf{x}_{t+1}). \quad (13)$$

In summary, the Pseudo-code of OLEF-KL is provided in Algorithm 1.

4. Theoretical analysis

This section derives the performance guarantee of the OLEF-KL algorithm. Specifically, two propositions are established. The first one discusses the case when using updating rule in Eq. (8) ($\rho = 1$), and the second one discusses the case when using updating rule in Eq. (9) ($\rho < 1$) based on the analysis of the former one.

4.1. Analysis when $\rho = 1$

Let the g represent an arbitrary classifier in the Hilbert space $\mathcal{H}_K$. The squared norm of g in $\mathcal{H}$, denoted as $H^2 = \|g\|_{\mathcal{H}_K}^2$, measures the length or magnitude of the hypothesis g in this space. The size of the data stream is T . We denote the number of occurrences of feature evolution as N , denote the time that the last feature evolution occurs as T_N and $T_N \ll T$. Denote $\mathcal{L}(f_t; \mathbf{x}_t, y_t)$ as $\mathcal{L}(f_t(\mathbf{x}_t))$ for convenience. Based on the above notations, we can establish the regret bound for the proposed method during the overall online learning procedure.

Proposition 1. Let $f_t, t \in \mathbb{T}$ be a sequence of classifiers generated by the updating rule in Eq. (8). If we assume that $K(\mathbf{x}_i, \mathbf{x}_j) \leq 1$, and the loss function $\mathcal{L}(f_t(\mathbf{x}_t))$ is L -Lipschitz, then for any $g_t \in \mathcal{H}$, the regret of the learned classifier can be bounded as follows:

$$\sum_{t \in \mathbb{T}} \mathcal{L}(f_t(\mathbf{x}_t)) - \sum_{t \in \mathbb{T}} \mathcal{L}(g_t(\mathbf{x}_t)) \leq \frac{H^2}{2\eta} + \eta L^2 T + N L H T_N + \eta N L^2 T_N^2. \quad (14)$$

If we set the coefficient as $\eta_t = 1/\sqrt{t}$, the algorithm achieves a sublinear regret $\mathcal{O}(\sqrt{T})$:

$$\sum_{t \in \mathbb{T}} \mathcal{L}(f_t(\mathbf{x}_t)) - \sum_{t \in \mathbb{T}} \mathcal{L}(g_t(\mathbf{x}_t)) \leq \mathcal{O}(\sqrt{T}) \quad (15)$$

Proof (Proof of Proposition 1).

The updating rule in Eq. (8) for f_t has two forms based on whether the feature evolution occurs or not at the iteration t . This proof considers two cases separately. Let the set $\{\mathcal{d}_{t-1} = \mathcal{d}_t | t \in \mathbb{T}\}$ as $\mathbb{T}_1$ and the set $\{\mathcal{d}_{t-1} \neq \mathcal{d}_t | t \in \mathbb{T}\}$ as $\mathbb{T}_2$. Clearly, we have $\mathbb{T} = \{\mathbb{T}_1 \cup \mathbb{T}_2\}$.

Case 1: $t \in \mathbb{T}_1$. In these trials, the updating rule in Eq. (8) for f_t can be rewrite as:

$$f_{t+1} = f_t - \eta \nabla \mathcal{L}(f_t(\mathbf{x}_t)) \mathcal{K}(\mathbf{x}_t, \cdot). \quad (16)$$

Thus, we have the following equation:

$$\begin{aligned}
&\|f_t - g_t\|_{\mathcal{H}_K}^2 - \|f_{t+1} - g_t\|_{\mathcal{H}_K}^2 \\
&= \|f_t - g_t\|_{\mathcal{H}_K}^2 - \|f_t - g_t - \eta \nabla \mathcal{L}(f_t(\mathbf{x}_t)) \mathcal{K}(\mathbf{x}_t, \cdot)\|_{\mathcal{H}_K}^2 \\
&= 2\eta \nabla \mathcal{L}(f_t(\mathbf{x}_t)) \langle \mathcal{K}(\mathbf{x}_t, \cdot), f_t - g_t \rangle - \|\eta \nabla \mathcal{L}(f_t(\mathbf{x}_t)) \mathcal{K}(\mathbf{x}_t, \cdot)\|_{\mathcal{H}_K}^2.
\end{aligned} \quad (17)$$

Using the reproducing property of $\mathcal{H}_K$, we have $\langle \mathcal{K}(\mathbf{x}_t, \cdot), f \rangle = f(\mathbf{x}_t)$. Recalling that we also have $\nabla \mathcal{L} \leq L$ and $\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) \leq 1$. Then, the above formulation can be written as:

$$\|f_t - g_t\|_{\mathcal{H}_K}^2 - \|f_{t+1} - g_t\|_{\mathcal{H}_K}^2 \geq 2\eta \nabla \mathcal{L}(f_t(\mathbf{x}_t)) [f_t(\mathbf{x}_t) - g_t(\mathbf{x}_t)] - \eta^2 L^2. \quad (18)$$

Rearrange it as:

$$\nabla \mathcal{L}(f_t(\mathbf{x}_t)) [f_t(\mathbf{x}_t) - g_t(\mathbf{x}_t)] \leq \frac{\|f_t - g_t\|_{\mathcal{H}_K}^2 - \|f_{t+1} - g_t\|_{\mathcal{H}_K}^2}{2\eta} + \frac{\eta L^2}{2}. \quad (19)$$

Due to the convexity of the loss function, we have:

$$\mathcal{L}(f_t(\mathbf{x}_t)) - \mathcal{L}(g_t(\mathbf{x}_t)) \leq \nabla \mathcal{L}(f_t(\mathbf{x}_t)) [f_t(\mathbf{x}_t) - g_t(\mathbf{x}_t)] \quad (20)$$

Summing over $|\mathbb{T}_1|$, we have the regret in the trials $t \in \mathbb{T}_1$:

$$\begin{aligned} \sum_{t \in \mathbb{T}_1} \{\mathcal{L}(f_t(\mathbf{x}_t)) - \mathcal{L}(g_t(\mathbf{x}_t))\} &\leq \sum_{t \in \mathbb{T}_1} \{\nabla \mathcal{L}(f_t(\mathbf{x}_t)) [f_t(\mathbf{x}_t) - g_t(\mathbf{x}_t)]\} \\ &\leq \frac{\sum_{t \in \mathbb{T}_1} \{\|f_t - g_t\|_{\mathcal{H}_K}^2 - \|f_{t+1} - g_t\|_{\mathcal{H}_K}^2\}}{2\eta} + \frac{\eta L^2}{2} |\mathbb{T}_1| \end{aligned} \quad (21)$$

Case 2: $t \in \mathbb{T}_2$. The updating rule for f_t includes three steps: (1) dimension adjustment of support vectors; (2) inclusion of new support vector; (3) Jaccard coefficient adaptation. In this situation, the updating rule in trials $|\mathbb{T}_2|$ can be expressed as:

$$\begin{aligned} f_t &= \sum_{\tau=1}^{t-1} \alpha_\tau J_{\tau,t} \mathcal{K}(\Pi_{\mathcal{A}_t} \mathbf{x}_\tau, \cdot), \\ f'_t &= f_t - \underbrace{\eta \nabla \mathcal{L}(f_t) \mathcal{K}(\mathbf{x}_t, \cdot)}_{\Phi_{A_t}}, \\ f_{t+1} &= f'_t + \underbrace{\sum_{\tau \in \mathbb{T} \setminus \{t\}} \eta \Delta J_{\tau,t} \nabla \mathcal{L}(f_\tau) \mathcal{K}(\Pi_{\mathcal{A}_t} \mathbf{x}_\tau, \cdot)}_{\Phi_{B_t}}, \end{aligned} \quad (22)$$

where $\Delta J_{\tau,t} = \Delta J_{\tau,t-1} - \Delta J_{\tau,t}$ is the Jaccard coefficient difference. Note that, it follows $0 \leq \Delta J_{\tau,t} < 1$ and $\sum_{t \in \mathbb{T}} \Delta J_{\tau,t} \leq 1$ for any $\tau \in \mathbb{T}$.

Based on the above updating rule and notations, the Eq. (17) has the form:

$$\begin{aligned} \|f_t - g_t\|_{\mathcal{H}_K}^2 - \|f_{t+1} - g_t\|_{\mathcal{H}_K}^2 &= \|f_t - g_t\|_{\mathcal{H}_K}^2 - \|f_t - g_t + \Phi_{B_t} - \Phi_{A_t}\|_{\mathcal{H}_K}^2 \\ &= 2 \langle f_t - g_t, \Phi_{A_t} - \Phi_{B_t} \rangle - \|\Phi_{B_t} - \Phi_{A_t}\|_{\mathcal{H}_K}^2. \end{aligned} \quad (23)$$

Using the reproducing property $\langle \mathcal{K}(\mathbf{x}_t, \cdot), f \rangle = f(\mathbf{x}_t)$, the first term of Eq. (23) is:

$$\langle f_t - g_t, \Phi_{A_t} - \Phi_{B_t} \rangle = \eta \nabla \mathcal{L}(f_t(\mathbf{x}_t)) [f_t(\mathbf{x}_t) - g_t(\mathbf{x}_t)] + \langle f_t - g_t, -\Phi_{B_t} \rangle. \quad (24)$$

Thus, rearranging Eq. (23) yields:

$$\begin{aligned} \nabla \mathcal{L}(f_t(\mathbf{x}_t)) [f_t(\mathbf{x}_t) - g_t(\mathbf{x}_t)] &= \frac{\|f_t - g_t\|_{\mathcal{H}_K}^2 - \|f_{t+1} - g_t\|_{\mathcal{H}_K}^2}{2\eta} \\ &\quad + \frac{2 \langle f_t - g_t, \Phi_{B_t} \rangle + \|\Phi_{B_t} - \Phi_{A_t}\|_{\mathcal{H}_K}^2}{2\eta} \end{aligned} \quad (25)$$

Using the convexity of loss function and summing over $|\mathbb{T}_2|$ achieve the regret in trials $\mathbb{T}_2$:

$$\begin{aligned} \sum_{t \in \mathbb{T}_2} \{\mathcal{L}(f_t(\mathbf{x}_t)) - \mathcal{L}(g_t(\mathbf{x}_t))\} &\leq \sum_{t \in \mathbb{T}_2} \{\nabla \mathcal{L}(f_t(\mathbf{x}_t)) [f_t(\mathbf{x}_t) - g_t(\mathbf{x}_t)]\} \\ &\leq \frac{\sum_{t \in \mathbb{T}_2} \{\|f_t - g_t\|_{\mathcal{H}_K}^2 - \|f_{t+1} - g_t\|_{\mathcal{H}_K}^2\}}{2\eta} \\ &\quad + \frac{\sum_{t \in \mathbb{T}_2} \{2 \langle f_t - g_t, \Phi_{B_t} \rangle + \|\Phi_{B_t} - \Phi_{A_t}\|_{\mathcal{H}_K}^2\}}{2\eta} \end{aligned} \quad (26)$$

Note that, since we assume that feature evolution occurs N times, we then have $|\mathbb{T}_2| = N$. Thus, we have:

$$\sum_{t \in \mathbb{T}_2} \langle f_t - g_t, \Phi_{B_t} \rangle = \sum_{t \in \mathbb{T}_2} \sum_{\tau \in \mathbb{T} \setminus \{t\}} \eta \Delta J_{\tau,t} \nabla \mathcal{L}(f_\tau(\mathbf{x}_\tau)) [f_t(\Pi_{\mathcal{A}_t} \mathbf{x}_\tau) - g_t(\Pi_{\mathcal{A}_t} \mathbf{x}_\tau)] \quad (27)$$

$$\leq \eta N L H \cdot \text{MAX}\{\mathbb{T}_2\}$$

$$\leq \eta N L H T_N,$$

where $T_N = \text{MAX}\{\mathbb{T}_2\}$ is the maximum number in the set $\mathbb{T}_2$, denoting the time the last feature evolution occurs. Meanwhile, we have:

$$\begin{aligned} \sum_{t \in \mathbb{T}_2} \|\Phi_{B_t} - \Phi_{A_t}\|_{\mathcal{H}_K}^2 &= \sum_{t \in \mathbb{T}_2} \left\| \sum_{\tau \in \mathbb{T} \setminus \{t\}} \eta \Delta J_{\tau,t} \nabla \mathcal{L}(f_\tau) \mathcal{K}(\Pi_{\mathcal{A}_t} \mathbf{x}_\tau, \cdot) - \eta \nabla \mathcal{L}(f_t) \mathcal{K}(\Pi_{\mathcal{A}_t} \mathbf{x}_t, \cdot) \right\|_{\mathcal{H}_K}^2 \\ &\leq \eta^2 N L^2 (\text{MAX}\{\mathbb{T}_2\})^2 \\ &\leq \eta^2 N L^2 T_N^2 \end{aligned} \quad (28)$$

Combining Eqs. (26), (27) and (28) yields:

$$\begin{aligned} \sum_{t \in \mathbb{T}_2} \{\mathcal{L}(f_t(\mathbf{x}_t)) - \mathcal{L}(g_t(\mathbf{x}_t))\} &\leq \frac{\sum_{t \in \mathbb{T}_2} \{\|f_t - g_t\|_{\mathcal{H}_K}^2 - \|f_{t+1} - g_t\|_{\mathcal{H}_K}^2\}}{2\eta} \\ &\quad + N L H T_N + \eta N L^2 T_N^2. \end{aligned} \quad (29)$$

Combing Eqs. (21) and (29) achieves the overall regret of the proposed algorithm:

$$\sum_{t \in \mathbb{T}} \mathcal{L}(f_t(\mathbf{x}_t)) - \sum_{t \in \mathbb{T}} \mathcal{L}(g_t(\mathbf{x}_t)) \leq \frac{H^2}{2\eta} + \eta L^2 T + N L H T_N + \eta N L^2 T_N^2. \quad (30)$$

If we set the coefficient as $\eta_t = 1/\sqrt{t}$:

$$\begin{aligned} \sum_{t \in \mathbb{T}} \mathcal{L}(f_t(\mathbf{x}_t)) - \sum_{t \in \mathbb{T}} \mathcal{L}(g_t(\mathbf{x}_t)) &\leq \frac{H^2}{2} \sqrt{T} + L^2 \sqrt{T} + N L H T_N + N L^2 T_N \\ &\leq \mathcal{O}(\sqrt{T}). \end{aligned} \quad (31)$$

This completes the proof. ■

Remark 2. In Proposition 1, setting the coefficient as $\eta_t = 1/\sqrt{t}$ achieves a regret in the order of $\mathcal{O}(\sqrt{T})$. This means that the cumulative loss disagreement across f_t and g_t is sublinear with respect to the time horizon. The sublinearity implies that the time average regret $\frac{\text{Regret}(T)}{T}$ is asymptotically non-positive as the number of data number goes to infinity. This suggests that, asymptotically, the learning model sequence incrementally generated by Algorithm 1 has performance no worse than any model g_t in $\mathcal{H}_K$.

Remark 3. From Proposition 1, we can see that the regret under evolving features is unsurprisingly related to N and T_N . Note that, we have $T_N \ll T$ as N is finite and T approaches to infinity. Therefore, the regret of the proposed algorithm still grows at a sublinear rate.

4.2. Analysis when using $\rho < 1$

Next, let us consider the updating rules with exponentially weighted decay strategy in Eq. (9) ($\rho < 1$). Based on the same notations and assumptions as previously introduced, we then establish the following regret bound.

Proposition 2. Using the same notations and assumptions in Proposition 1, when $\rho < 1$, the regret of the learned classifier generated by Eq. (9) is sublinear ($\mathcal{O}(\sqrt{T})$) by setting ρ_t as $1 - \frac{1}{\sqrt{t}}$ and η_t as $\frac{1}{\sqrt{t}}$.

$$\sum_{t \in \mathbb{T}} \mathcal{L}(f_t(\mathbf{x}_t)) - \sum_{t \in \mathbb{T}} \mathcal{L}(g_t(\mathbf{x}_t)) \leq \mathcal{O}(\sqrt{T}). \quad (32)$$

Table 1

Details of datasets. $\mathbb{D}_{orig}$, $\mathbb{D}_{rand}$ and $\mathbb{D}_{new}$ are the feature set in the original, random and new feature space, respectively.

	#Samples	$ \mathbb{D}_{orig} $	$ \mathbb{D}_{rand} $	$ \mathbb{D}_{new} $
HTRU2	17 898	8	4	12
TUANDROMD	4464	240	120	30
credit-a	653	15	7	22
dna	949	180	90	30
ijcnn1	49 990	22	11	30
magic04	19 020	15	5	15
mushroom	8124	21	10	30
musk2	6598	166	83	30
phishing	11 055	68	34	30
rna	59 535	8	4	12
sensors	10 638	48	24	30
spambase	4601	57	28	30
splice	3175	60	30	30
svmguide1	7089	4	2	6
wdbc	569	30	15	30

The proofs of this proposition follows quite readily from the previous one. Due to the limitation of page length, we provided the detailed proof in the *supplementary material*.¹

Remark 4. Compared with the regret in Proposition 1, the regret when setting $\rho < 1$ incurs extra regret. If one sets ρ_i as $1 - \frac{1}{\sqrt{i}}$ and η_i as $\frac{1}{\sqrt{i}}$, the overall regret when using the exponentially weighted decay strategy ($\rho < 1$) also can be sublinear with respect to the time horizon. Sublinearity suggests that asymptotically, the sequence of the learning model generated incrementally by Algorithm 1 when $\rho < 1$ has performance not worse than any model g_t in $\mathcal{H}_K$.

5. Experimental results

In this section, we conduct extensive experiments to evaluate the performance of our OLEF-KL algorithm. We show that OLEF-KL consistently achieves competitive performance under various feature evolution scenarios. Due to the limitation of the page, we only provided summarized results here. Readers may find more detailed results in the *supplementary material*. We published our code and detailed experimental results here.¹

5.1. General settings

5.1.1. Public datasets

We selected a set of 15 publicly available datasets from the UCI Repository. The selection process was executed with a certain level of randomness, ensuring that our dataset collection is representative of diverse sizes and domains. A comprehensive description of these 15 binary-class datasets can be found in Table 1. Samples in each dataset were manually arranged in a sequential manner to simulate a stream, following most previous studies [23,26,30]. In particular, to simulate feature evolution, we have drawn upon prior research [8] to guide our settings of feature evolution. Specifically, we generated a random feature set, denoted as $\mathbb{D}_{rand}$, which is viewed as disappeared features. At the same time, we mapped the original data to an alternative feature space $\mathbb{D}_{new}$ by applying random Gaussian matrices. Consequently, we obtained appearing features. For details of the settings of feature evolution, we direct the reader to the following subsections.

5.1.2. Implementation details

For a fair comparison, the experiment was conducted based on the strategy established in previous studies, such as [23,26,30]. We use the Test-Then-Train strategy for online learning. Namely, each algorithm was presented with an unlabeled sample and made a prediction. Subsequently, the true label was received for the update of models. We performed a total of 20 independent runs for each dataset and each algorithm. The results were presented as the average performance metrics, accompanied by their respective standard deviations. Performance evaluation was performed through accuracy measurement (ACC), area under the curve (AUC) and F1 Score (F1). Meanwhile, to conduct a more rigorous assessment of relative performance across different methods, we employed the Nemenyi test on the rankings.

5.1.3. Parameters of OLEF-KL

There are two crucial parameters η , ρ needing to be determined in OLEF-KL. Empirically, we have set η at a value of 1 and ρ to 0.999 as our initial configuration. Unless otherwise specified, we used the Gaussian kernel function in the proposed method, denoted as $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / \sigma)$, with σ assigned a value of 0.2. The comprehensive exploration of parameter sensitivity will be addressed in forthcoming subsections.

5.2. A toy example

To provide a demonstrative illustration of the effectiveness of OLEF-KL, we devised two toy examples, denoted as 2-cube and 2-moon, situated within a three-dimensional space. These toy examples are visually represented in Fig. 2. The samples in these toy datasets have been manually arranged in a sequential manner, simulating data streams. We divided the data streams into two distinct periods. During the initial period, the dataset comprises features X and Y, which are inherently inseparable within this feature space. Then, the subsequent period introduces a notable transition, with feature X disappearing while feature Z appearing. In particular, this newly formed feature space renders samples separable, thus altering the patterns of the dataset. We report the classification results for the second period, as shown in Fig. 2(d) and Fig. 2(h). As the results, the proposed OLEF-KL achieved satisfied performance, which only made 1 and 2 mistakes in datasets 2-cube and 2-moon, respectively. This fact visually shows the effectiveness of OLEF-KL in adapting to the evolving feature space. Meanwhile, the satisfied performance in example 2-moon further verifies the effectiveness of OLEF-KL in discovering nonlinear patterns.

5.3. Comparison under general evolution

This section proceeds to evaluate the performance of OLEF-KL under general evolution scenario, which is visually shown in Fig. 1(e).

5.3.1. Scenario settings

The simulation of general evolving feature data streams was conducted as follows. Each dataset comprises two sets of features: the random feature set and the original feature set. Particularly, to simulate feature disappearance, random features were sequentially removed, and to simulate feature appearance, original features were introduced across S periods, where S takes on values from the set $\{2, 3, 4, 5\}$. For instance, when $S = 2$, half of the features from $\mathbb{D}_{orig}$ were provided in the first period, and the remaining features were introduced in the subsequent period. Concurrently, all features from $\mathbb{D}_{rand}$ were initially available, but they were systematically removed throughout the entire process until none of them remained. The illustration of this setting can be found in Figure S1 in the *supplementary material*.

¹ <https://github.com/Kan9594/One-Pass-Online-Learning-under-Evolving-Feature-Data-Streams-A-Non-Parametric-Model>

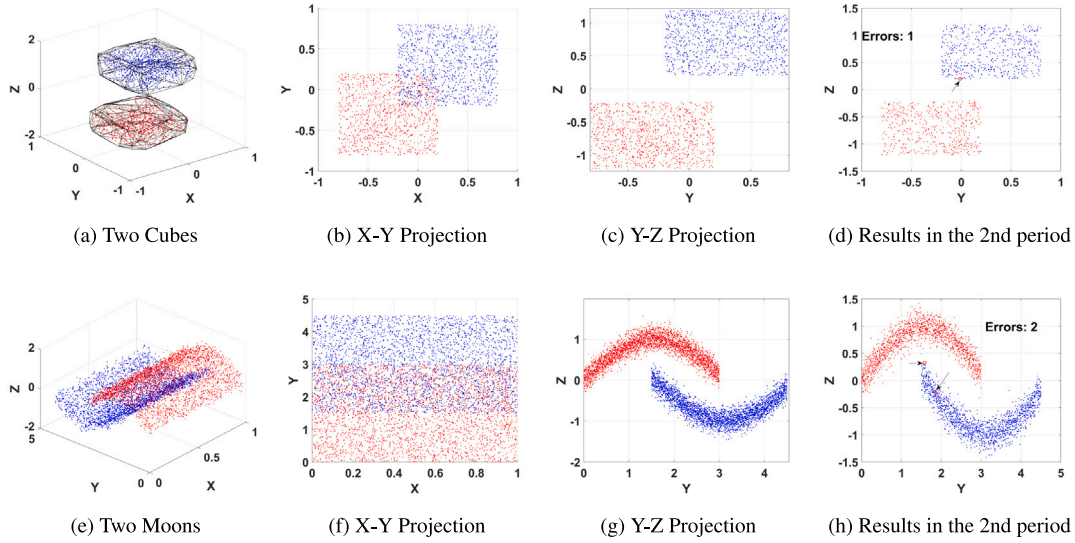


Fig. 2. Learning results on toy examples. Red and blue dots denote different classes. (a) and (e): Illustration of 2-cube and 2-moon datasets, respectively; (b) and (f) data in 1st period: the X-Y feature space; (c) and (g) data in 2nd period: the Y-Z feature space; (d) and (h) the performance of OLEF-KL in the 2nd period. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

5.3.2. Competitor settings

We compared the proposed OLEF-KL with eight algorithms in this scenario. **NPA** [13] and **NOGD** [11] represent traditional one-pass online learning algorithms. In our setup, when the feature space undergoes changes, both NOGD and NPA are initiated from the beginning. **OLSF** [7] employs a passive-aggressive optimization method, and we also make slight extensions (**OLSF_{OGD}**) by incorporating gradient descent optimization method. **OLVF** [38] deals with the evolving features by dynamically projecting the classifier and the training samples onto their shared feature subspace. **OLCF** [4] maps the heterogeneous feature in a shared subspace in an online manner. **OLI2DS** [39], an extension of [38], replaces the projection weights by feature uncertainty. **RSOL** [41] considers the robustness and sparsity during online learning the evolving features.

5.3.3. Comparison results

We provide the detailed classification results under 5-period general evolution scenario in Table 2, in terms of Acc, AUC and F1 Score. More detailed results under different periods can be found in Table S2 and Table S4 in the *supplementary material*. Notably, our proposed OLEF-KL algorithm exhibits superior performance relative to other methods in most cases, particularly with regard to ACC. This enhanced performance can be attributed to the inherent flexibility of our non-parametric model. Our non-parametric algorithm refrains from imposing strong assumptions regarding the functional form of the classifier and the underlying distribution of data streams. Consequently, it excels at modeling nonlinear relationships within data streams. Therefore, our algorithm is particularly advantageous in scenarios where the true distribution of data or features remains unknown. In terms of AUC and F1 Score performance, our algorithm falls slightly short of OLI2DS, particularly in datasets such as *ijcnn*, *musk2*, *rna*, and *spambase*. This performance difference can be attributed to the design of a cost-sensitive loss function in OLI2DS to address imbalanced data streams. It should be noted that our research focuses primarily on the development of algorithms to learn from evolving feature data streams, and addressing data imbalance falls outside the scope of this study. However, this area presents an interesting topic for future research.

5.4. Comparison under synchronized evolution

Within this subsection, our objective is to assess the performance of OLEF-KL under synchronized evolving scenarios, which is visually represented in Fig. 1(d).

5.4.1. Scenario settings

The simulation of synchronized evolving feature data streams was conducted as follows. For each dataset, it was systematically partitioned into a predetermined number of periods denoted as S , where S takes the values from the set $\{2, 3, 4, 5\}$. In the initial period, we introduced a combination of features sourced from 100% of $\mathbb{D}_{rand}$, and features originating from $\frac{100}{S}\%$ of $\mathbb{D}_{orig}$. Subsequently, in the second period, a dynamic adjustment was made. Specifically, $\frac{100}{S-1}\%$ of the features from $\mathbb{D}_{rand}$ were removed. Meanwhile, an analogous proportion of $\frac{100}{S-1}\%$ of features from $\mathbb{D}_{orig}$, distinct from those present in the first period, was introduced. These deliberate adjustments were made to stimulate the phenomenon of feature synchronized disappearance and appearance. The illustration of this setting can be found in Figure S2 in the *supplementary material*.

5.4.2. Competitor settings

We compared the proposed OLEF-KL with **NOGD**, **NPA**, **OLSF_{OGD}**, **OLSF** and two additional methods. **FF**: First Features (FF) learns a linear classifier based on features received in the first period over all the learning procedure. **OPID** [10], also a naive model for work [37], is designed for the synchronized evolution, by compressing information of disappeared features, and expanding to include the appeared features.

5.4.3. Comparison results

The detailed comparison results in terms of ACC, AUC and F1 Score are provided in Table 3. Specifically, it provides results under 5-period synchronized evolution scenarios. Readers can find more results on multi-period in Table S5–S7 in the *supplementary material*. As can be seen from the results, our proposed OLEF-KL algorithm exhibits superior performance relative to other methods in most cases. In comparison to its foremost competitors (OPID), OLEF-KL attains a record of 37 victories, 2 draws, and 6 loss. This enhanced performance can be attributed to the non-linearity learnability of our non-parametric model. Readers may also find the comparison bar plots (Figure S13–S15) and error curves (Figure S16), along with their analysis, in the *supplementary material*. Observations in these results also underscore the effectiveness of OLEF-KL in learning synchronized evolving feature data streams.

Table 2

Performance comparison under 5-period general evolution scenarios, in term of ACC, AUC and F1 Score (mean $\pm$ std). $\bullet/\circ$ indicate that OLEF-KL is significantly better/worse than competitors (hypothesis supported by paired t-tests at 0.05 significant level). The best results are indicated in **bold**.

Metric	Datasets	NOGD	NPA	OLSF _{OGD}	OLSF	OLVF	OLCF	OLI2DS	RSOL	OLEF-KL
ACC	HTRU2	0.948 ± 0.001•	0.935 ± 0.001•	0.959 ± 0.001•	0.938 ± 0.001•	0.961 ± 0.001•	0.961 ± 0.002•	0.858 ± 0.006•	0.919 ± 0.008•	0.964 ± 0.001
	TUANDROMD	0.809 ± 0.004•	0.782 ± 0.006•	0.870 ± 0.003•	0.879 ± 0.004•	0.900 ± 0.004	0.858 ± 0.005•	0.861 ± 0.006•	0.836 ± 0.031•	0.899 ± 0.003
	credit-a	0.608 ± 0.019•	0.611 ± 0.015•	0.653 ± 0.017•	0.634 ± 0.015•	0.656 ± 0.021•	0.641 ± 0.026•	0.641 ± 0.023•	0.635 ± 0.022•	0.666 ± 0.012
	dna	0.573 ± 0.020•	0.606 ± 0.012•	0.758 ± 0.011	0.778 ± 0.010⊙	0.782 ± 0.010⊙	0.707 ± 0.010•	0.782 ± 0.014⊙	0.765 ± 0.010	0.761 ± 0.016
	ijcnn1	0.906 ± 0.001•	0.836 ± 0.001•	0.907 ± 0.001•	0.836 ± 0.001•	0.905 ± 0.001•	0.904 ± 0.001•	0.447 ± 0.007•	0.859 ± 0.003•	0.909 ± 0.001
	magic04	0.686 ± 0.002•	0.600 ± 0.003•	0.713 ± 0.001⊙	0.604 ± 0.003•	0.685 ± 0.003•	0.708 ± 0.003•	0.532 ± 0.007•	0.629 ± 0.005•	0.711 ± 0.003
	mushroom	0.858 ± 0.002•	0.831 ± 0.004•	0.888 ± 0.002•	0.841 ± 0.003•	0.893 ± 0.003•	0.878 ± 0.004•	0.892 ± 0.002•	0.861 ± 0.005•	0.953 ± 0.002
	musk2	0.754 ± 0.004•	0.733 ± 0.004•	0.836 ± 0.004•	0.804 ± 0.004•	0.841 ± 0.004•	0.822 ± 0.005•	0.671 ± 0.011•	0.821 ± 0.005•	0.906 ± 0.003
	phishing	0.748 ± 0.002•	0.787 ± 0.003•	0.867 ± 0.001•	0.848 ± 0.003•	0.879 ± 0.002•	0.871 ± 0.003•	0.870 ± 0.004•	0.880 ± 0.003•	0.883 ± 0.002
	rna	0.811 ± 0.001•	0.782 ± 0.002•	0.852 ± 0.001⊙	0.795 ± 0.002•	0.844 ± 0.001•	0.828 ± 0.002•	0.790 ± 0.002•	0.828 ± 0.003•	0.849 ± 0.001
	sensors	0.964 ± 0.002•	0.986 ± 0.001•	0.998 ± 0.000⊙	0.999 ± 0.000⊙	0.999 ± 0.000⊙	0.995 ± 0.001•	0.999 ± 0.000⊙	0.999 ± 0.000⊙	0.998 ± 0.000
	spambase	0.779 ± 0.005•	0.760 ± 0.006•	0.847 ± 0.003•	0.789 ± 0.004•	0.845 ± 0.005•	0.807 ± 0.004•	0.820 ± 0.006•	0.808 ± 0.016•	0.855 ± 0.002
	splice	0.521 ± 0.009•	0.540 ± 0.007•	0.632 ± 0.008•	0.643 ± 0.007•	0.658 ± 0.007•	0.637 ± 0.008•	0.657 ± 0.005•	0.683 ± 0.006•	0.693 ± 0.006
	svmguidel	0.702 ± 0.004•	0.624 ± 0.006•	0.708 ± 0.003•	0.624 ± 0.006•	0.695 ± 0.004•	0.707 ± 0.005•	0.668 ± 0.007•	0.475 ± 0.016•	0.766 ± 0.004
	wdbc	0.604 ± 0.013•	0.602 ± 0.017•	0.759 ± 0.015•	0.789 ± 0.015•	0.802 ± 0.014•	0.794 ± 0.015•	0.744 ± 0.022•	0.825 ± 0.012•	0.839 ± 0.010
AUC	HTRU2	0.666 ± 0.010•	0.784 ± 0.007•	0.747 ± 0.011•	0.796 ± 0.007•	0.812 ± 0.008•	0.797 ± 0.016•	0.840±0.007⊙	0.878 ± 0.007⊙	0.831 ± 0.008
	TUANDROMD	0.599 ± 0.010•	0.668 ± 0.009•	0.769 ± 0.006•	0.851 ± 0.006•	0.866 ± 0.006	0.752 ± 0.016•	0.856 ± 0.007•	0.820 ± 0.023•	0.865 ± 0.005
	credit-a	0.613 ± 0.020•	0.612 ± 0.018•	0.670 ± 0.027	0.636 ± 0.018•	0.664 ± 0.021•	0.645 ± 0.026•	0.630 ± 0.039•	0.640 ± 0.023•	0.674 ± 0.016
	dna	0.580 ± 0.022•	0.616 ± 0.013•	0.758 ± 0.040•	0.782 ± 0.013	0.785 ± 0.016	0.716 ± 0.024•	0.783 ± 0.025	0.772 ± 0.016	0.779 ± 0.026
	ijcnn1	0.505 ± 0.004•	0.514±0.005⊙	0.505 ± 0.004•	0.514 ± 0.005⊙	0.507 ± 0.005⊙	0.504 ± 0.003•	0.582±0.005⊙	0.512±0.004⊙	0.507 ± 0.004
	magic04	0.608 ± 0.004•	0.568 ± 0.004•	0.656 ± 0.003•	0.574 ± 0.004•	0.635 ± 0.003•	0.642 ± 0.005•	0.567 ± 0.006•	0.613 ± 0.005•	0.676 ± 0.004
	mushroom	0.863 ± 0.003•	0.827 ± 0.005•	0.895 ± 0.003•	0.837 ± 0.005•	0.894 ± 0.003•	0.882 ± 0.005•	0.895 ± 0.005•	0.855 ± 0.006•	0.947 ± 0.003
	musk2	0.527 ± 0.009•	0.594 ± 0.008•	0.592 ± 0.011•	0.689 ± 0.007⊙	0.664 ± 0.009⊙	0.560 ± 0.012•	0.779 ± 0.008⊙	0.722±0.009⊙	0.649 ± 0.014
	phishing	0.756 ± 0.004•	0.789 ± 0.003•	0.887 ± 0.004•	0.850 ± 0.003•	0.891 ± 0.003•	0.886 ± 0.003•	0.874 ± 0.008•	0.887 ± 0.003•	0.898 ± 0.002
	rna	0.723 ± 0.002•	0.736 ± 0.003•	0.786 ± 0.002•	0.750 ± 0.002•	0.791 ± 0.002•	0.767 ± 0.003•	0.836 ± 0.002⊙	0.780 ± 0.005•	0.797 ± 0.002
	sensors	0.963 ± 0.003•	0.986 ± 0.001•	0.998 ± 0.001	0.999 ± 0.000⊙	0.999±0.001⊙	0.992 ± 0.002•	0.999 ± 0.000⊙	0.999 ± 0.001⊙	0.998 ± 0.000
	spambase	0.703 ± 0.009•	0.732 ± 0.007•	0.788 ± 0.007•	0.765 ± 0.006•	0.804 ± 0.008	0.729 ± 0.009•	0.824 ± 0.006⊙	0.802 ± 0.018	0.804 ± 0.005
	splice	0.521 ± 0.009•	0.541 ± 0.008•	0.645 ± 0.012•	0.645 ± 0.007•	0.664 ± 0.007•	0.650 ± 0.014•	0.654 ± 0.014•	0.681 ± 0.009•	0.706 ± 0.011
	svmguidel	0.707 ± 0.006•	0.621 ± 0.007•	0.713 ± 0.005•	0.622 ± 0.007•	0.691 ± 0.005•	0.710 ± 0.007•	0.653 ± 0.011•	0.509 ± 0.007•	0.783 ± 0.004
	wdbc	0.524 ± 0.016•	0.565 ± 0.022•	0.595 ± 0.029•	0.751 ± 0.018	0.710 ± 0.024•	0.682 ± 0.036•	0.790±0.030⊙	0.798 ± 0.016⊙	0.754 ± 0.019
F1	HTRU2	0.628 ± 0.010•	0.648 ± 0.007•	0.738 ± 0.009•	0.664 ± 0.007•	0.768 ± 0.006•	0.765 ± 0.014•	0.522 ± 0.011•	0.654 ± 0.022•	0.788 ± 0.007
	TUANDROMD	0.891 ± 0.002•	0.866 ± 0.004•	0.924 ± 0.002•	0.924 ± 0.002•	0.940 ± 0.002	0.917 ± 0.003•	0.911 ± 0.004•	0.891 ± 0.024•	0.940 ± 0.002
	credit-a	0.668 ± 0.026•	0.648 ± 0.013•	0.705 ± 0.034	0.665 ± 0.014•	0.692 ± 0.021•	0.685 ± 0.033•	0.633 ± 0.046•	0.659 ± 0.028•	0.715 ± 0.013
	dna	0.599 ± 0.036•	0.633 ± 0.010•	0.759 ± 0.024•	0.783 ± 0.010	0.786 ± 0.011	0.721 ± 0.021•	0.785 ± 0.016	0.763 ± 0.014•	0.788 ± 0.013
	ijcnn1	0.112 ± 0.013•	0.174±0.005⊙	0.109 ± 0.013•	0.174±0.005⊙	0.153 ± 0.008⊙	0.035 ± 0.020•	0.200 ± 0.002⊙	0.170±0.006⊙	0.143 ± 0.009
	magic04	0.800±0.001⊙	0.691 ± 0.002•	0.811 ± 0.001⊙	0.694 ± 0.002•	0.781 ± 0.002•	0.808±0.001⊙	0.524 ± 0.014•	0.705 ± 0.006•	0.794 ± 0.002
	mushroom	0.849 ± 0.002•	0.824 ± 0.004•	0.883 ± 0.002•	0.835 ± 0.004•	0.888 ± 0.003•	0.872 ± 0.004•	0.887 ± 0.003•	0.853 ± 0.005•	0.950 ± 0.002
	musk2	0.272 ± 0.011•	0.340 ± 0.009•	0.449 ± 0.014•	0.497 ± 0.008•	0.521 ± 0.011•	0.385 ± 0.021•	0.458 ± 0.009•	0.545 ± 0.010•	0.568 ± 0.017
	phishing	0.798 ± 0.002•	0.810 ± 0.003•	0.886 ± 0.001•	0.863 ± 0.003•	0.895 ± 0.002•	0.889 ± 0.002•	0.883 ± 0.004•	0.894 ± 0.002•	0.900 ± 0.001
	rna	0.671 ± 0.002•	0.672 ± 0.003•	0.767 ± 0.001⊙	0.692 ± 0.002•	0.759 ± 0.002•	0.724 ± 0.004•	0.737 ± 0.002•	0.729 ± 0.005•	0.765 ± 0.002
	sensors	0.964 ± 0.002•	0.986 ± 0.001•	0.998 ± 0.000⊙	0.999 ± 0.000⊙	0.999 ± 0.000⊙	0.995 ± 0.001•	0.999 ± 0.000⊙	0.999 ± 0.000⊙	0.998 ± 0.000
	spambase	0.680 ± 0.011•	0.693 ± 0.007•	0.791 ± 0.005•	0.734 ± 0.006•	0.795 ± 0.007•	0.725 ± 0.008•	0.786 ± 0.005•	0.767 ± 0.015•	0.797 ± 0.004
	splice	0.566 ± 0.020•	0.559 ± 0.007•	0.658 ± 0.016•	0.656 ± 0.007•	0.673 ± 0.008•	0.659 ± 0.023•	0.654 ± 0.015•	0.688 ± 0.008•	0.708 ± 0.012
	svmguidel	0.776 ± 0.003•	0.663 ± 0.006•	0.779 ± 0.003•	0.662 ± 0.006•	0.756 ± 0.003•	0.774 ± 0.005•	0.699 ± 0.012•	0.333 ± 0.093•	0.801 ± 0.003
	wdbc	0.292 ± 0.051•	0.474 ± 0.021•	0.551 ± 0.040•	0.714 ± 0.021•	0.703 ± 0.023•	0.663 ± 0.040•	0.709 ± 0.013•	0.769 ± 0.013⊙	0.757 ± 0.019
win-tie-lose	44-0-1	43-0-2	35-4-6	35-3-7	32-6-7	44-0-1	32-2-11	33-3-9	-	

Table 3

Performance comparison under 5-period synchronized evolution scenarios, in term of ACC and AUC (mean $\pm$ std). $\star/\odot$ indicate that OLEF-KL is significantly better/worse than competitors (hypothesis supported by paired t-tests at 0.05 significant level). The best results are indicated in **bold**.

Metric	Datasets	FF	NOGD	NPA	OLSF _{OGD}	OLSF	OPID	OLEF-KL
ACC	HTRU2	0.255 $\pm$ 0.001 $\star$	0.946 $\pm$ 0.001 $\star$	0.933 $\pm$ 0.001 $\star$	0.950 $\pm$ 0.001 $\star$	0.934 $\pm$ 0.001 $\star$	0.884 $\pm$ 0.001 $\star$	0.966 $\pm$ 0.001
	TUANDROMD	0.798 $\pm$ 0.000 $\star$	0.824 $\pm$ 0.002 $\star$	0.810 $\pm$ 0.004 $\star$	0.845 $\pm$ 0.002 $\star$	0.819 $\pm$ 0.004 $\star$	0.755 $\pm$ 0.006 $\star$	0.893 $\pm$ 0.003
	credit-a	0.541 $\pm$ 0.008 $\star$	0.581 $\pm$ 0.012 $\star$	0.565 $\pm$ 0.015 $\star$	0.595 $\pm$ 0.013	0.563 $\pm$ 0.016 $\star$	0.598 $\pm$ 0.015	0.593 $\pm$ 0.017
	dna	0.513 $\pm$ 0.007 $\star$	0.584 $\pm$ 0.016 $\star$	0.635 $\pm$ 0.017 $\star$	0.621 $\pm$ 0.014 $\star$	0.642 $\pm$ 0.016$\odot$	0.602 $\pm$ 0.012 $\star$	0.634 $\pm$ 0.017
	ijcnn1	0.258 $\pm$ 0.001 $\star$	0.906 $\pm$ 0.001 $\star$	0.833 $\pm$ 0.001 $\star$	0.907 $\pm$ 0.001 $\star$	0.833 $\pm$ 0.001 $\star$	0.859 $\pm$ 0.001 $\star$	0.908 $\pm$ 0.001
	magic04	0.648 $\pm$ 0.000 $\star$	0.680 $\pm$ 0.002$\odot$	0.575 $\pm$ 0.003 $\star$	0.681 $\pm$ 0.002$\odot$	0.576 $\pm$ 0.003 $\star$	0.635 $\pm$ 0.004 $\star$	0.661 $\pm$ 0.003
	mushroom	0.488 $\pm$ 0.005 $\star$	0.740 $\pm$ 0.004 $\star$	0.692 $\pm$ 0.006 $\star$	0.741 $\pm$ 0.004 $\star$	0.692 $\pm$ 0.006 $\star$	0.605 $\pm$ 0.006 $\star$	0.828 $\pm$ 0.003
	musk2	0.293 $\pm$ 0.003 $\star$	0.811 $\pm$ 0.003 $\star$	0.794 $\pm$ 0.004 $\star$	0.830 $\pm$ 0.003 $\star$	0.800 $\pm$ 0.003 $\star$	0.801 $\pm$ 0.004 $\star$	0.901 $\pm$ 0.002
	phishing	0.556 $\pm$ 0.001 $\star$	0.792 $\pm$ 0.004 $\star$	0.765 $\pm$ 0.003 $\star$	0.798 $\pm$ 0.003 $\star$	0.770 $\pm$ 0.004 $\star$	0.614 $\pm$ 0.004 $\star$	0.817 $\pm$ 0.003
	rna	0.399 $\pm$ 0.001 $\star$	0.780 $\pm$ 0.002 $\star$	0.719 $\pm$ 0.001 $\star$	0.792 $\pm$ 0.001 $\star$	0.722 $\pm$ 0.001 $\star$	0.655 $\pm$ 0.002 $\star$	0.798 $\pm$ 0.001
	sensors	0.500 $\pm$ 0.003 $\star$	0.901 $\pm$ 0.002 $\star$	0.899 $\pm$ 0.003 $\star$	0.903 $\pm$ 0.002 $\star$	0.899 $\pm$ 0.003 $\star$	0.601 $\pm$ 0.004 $\star$	0.906 $\pm$ 0.003
	spambase	0.435 $\pm$ 0.005 $\star$	0.776 $\pm$ 0.005 $\star$	0.722 $\pm$ 0.005 $\star$	0.782 $\pm$ 0.004 $\star$	0.723 $\pm$ 0.005 $\star$	0.630 $\pm$ 0.007 $\star$	0.809 $\pm$ 0.004
	splice	0.517 $\pm$ 0.003 $\star$	0.518 $\pm$ 0.006 $\star$	0.537 $\pm$ 0.006 $\star$	0.523 $\pm$ 0.007 $\star$	0.540 $\pm$ 0.007 $\star$	0.604 $\pm$ 0.006$\odot$	0.588 $\pm$ 0.008
	svmguide1	0.561 $\pm$ 0.006 $\star$	0.721 $\pm$ 0.005 $\star$	0.662 $\pm$ 0.005 $\star$	0.722 $\pm$ 0.005 $\star$	0.663 $\pm$ 0.005 $\star$	0.656 $\pm$ 0.005 $\star$	0.816 $\pm$ 0.004
	wdbc	0.420 $\pm$ 0.025 $\star$	0.632 $\pm$ 0.013 $\star$	0.672 $\pm$ 0.014 $\star$	0.687 $\pm$ 0.015 $\star$	0.719 $\pm$ 0.015 $\star$	0.633 $\pm$ 0.015 $\star$	0.783 $\pm$ 0.015
AUC	HTRU2	0.505 $\pm$ 0.004 $\star$	0.630 $\pm$ 0.009 $\star$	0.798 $\pm$ 0.005 $\star$	0.648 $\pm$ 0.008 $\star$	0.802 $\pm$ 0.005 $\star$	0.607 $\pm$ 0.006 $\star$	0.843 $\pm$ 0.006
	TUANDROMD	0.507 $\pm$ 0.005 $\star$	0.613 $\pm$ 0.008 $\star$	0.743 $\pm$ 0.007 $\star$	0.688 $\pm$ 0.008 $\star$	0.761 $\pm$ 0.005 $\star$	0.656 $\pm$ 0.009 $\star$	0.818 $\pm$ 0.007
	credit-a	0.515 $\pm$ 0.012 $\star$	0.581 $\pm$ 0.015 $\star$	0.563 $\pm$ 0.017 $\star$	0.601 $\pm$ 0.016$\odot$	0.561 $\pm$ 0.019 $\star$	0.600 $\pm$ 0.019	0.591 $\pm$ 0.020
	dna	0.517 $\pm$ 0.012 $\star$	0.593 $\pm$ 0.018 $\star$	0.638 $\pm$ 0.021	0.642 $\pm$ 0.030	0.643 $\pm$ 0.019	0.604 $\pm$ 0.018 $\star$	0.636 $\pm$ 0.027
	ijcnn1	0.503 $\pm$ 0.003 $\star$	0.505 $\pm$ 0.002 $\star$	0.515 $\pm$ 0.004 $\odot$	0.505 $\pm$ 0.002 $\star$	0.515 $\pm$ 0.004 $\odot$	0.536 $\pm$ 0.005$\odot$	0.507 $\pm$ 0.003
	magic04	0.503 $\pm$ 0.002 $\star$	0.598 $\pm$ 0.004 $\star$	0.542 $\pm$ 0.003 $\star$	0.601 $\pm$ 0.004 $\star$	0.543 $\pm$ 0.003 $\star$	0.623 $\pm$ 0.003 $\odot$	0.621 $\pm$ 0.003
	mushroom	0.503 $\pm$ 0.002 $\star$	0.739 $\pm$ 0.006 $\star$	0.685 $\pm$ 0.006 $\star$	0.738 $\pm$ 0.006 $\star$	0.685 $\pm$ 0.006 $\star$	0.603 $\pm$ 0.006 $\star$	0.809 $\pm$ 0.004
	musk2	0.506 $\pm$ 0.005 $\star$	0.511 $\pm$ 0.007 $\star$	0.639 $\pm$ 0.006 $\star$	0.541 $\pm$ 0.007 $\star$	0.656 $\pm$ 0.006$\odot$	0.580 $\pm$ 0.008 $\star$	0.639 $\pm$ 0.009
	phishing	0.504 $\pm$ 0.003 $\star$	0.835 $\pm$ 0.005 $\star$	0.770 $\pm$ 0.003 $\star$	0.842 $\pm$ 0.005$\odot$	0.775 $\pm$ 0.003 $\star$	0.618 $\pm$ 0.003 $\star$	0.836 $\pm$ 0.004
	rna	0.502 $\pm$ 0.001 $\star$	0.723 $\pm$ 0.003 $\star$	0.684 $\pm$ 0.002 $\star$	0.746 $\pm$ 0.003 $\star$	0.687 $\pm$ 0.002 $\star$	0.592 $\pm$ 0.003 $\star$	0.766 $\pm$ 0.002
	sensors	0.505 $\pm$ 0.003 $\star$	0.901 $\pm$ 0.013 $\star$	0.899 $\pm$ 0.002 $\star$	0.903 $\pm$ 0.013	0.899 $\pm$ 0.002 $\star$	0.600 $\pm$ 0.004 $\star$	0.906 $\pm$ 0.006
	spambase	0.507 $\pm$ 0.004 $\star$	0.678 $\pm$ 0.012 $\star$	0.692 $\pm$ 0.007 $\star$	0.699 $\pm$ 0.009 $\star$	0.695 $\pm$ 0.007 $\star$	0.599 $\pm$ 0.009 $\star$	0.764 $\pm$ 0.007
	splice	0.508 $\pm$ 0.005 $\star$	0.517 $\pm$ 0.006 $\star$	0.538 $\pm$ 0.007 $\star$	0.524 $\pm$ 0.006 $\star$	0.540 $\pm$ 0.008 $\star$	0.606 $\pm$ 0.007$\odot$	0.594 $\pm$ 0.007
	svmguide1	0.507 $\pm$ 0.004 $\star$	0.661 $\pm$ 0.011 $\star$	0.638 $\pm$ 0.006 $\star$	0.665 $\pm$ 0.008 $\star$	0.639 $\pm$ 0.006 $\star$	0.655 $\pm$ 0.007 $\star$	0.798 $\pm$ 0.005
	wdbc	0.521 $\pm$ 0.015 $\star$	0.520 $\pm$ 0.015 $\star$	0.634 $\pm$ 0.020 $\star$	0.531 $\pm$ 0.025 $\star$	0.678 $\pm$ 0.018 $\star$	0.590 $\pm$ 0.019 $\star$	0.690 $\pm$ 0.023
F1	HTRU2	0.164 $\pm$ 0.001 $\star$	0.600 $\pm$ 0.011 $\star$	0.651 $\pm$ 0.006 $\star$	0.639 $\pm$ 0.008 $\star$	0.654 $\pm$ 0.006 $\star$	0.356 $\pm$ 0.008 $\star$	0.796 $\pm$ 0.007
	TUANDROMD	0.888 $\pm$ 0.000 $\star$	0.900 $\pm$ 0.001 $\star$	0.881 $\pm$ 0.002 $\star$	0.910 $\pm$ 0.001 $\star$	0.887 $\pm$ 0.002 $\star$	0.846 $\pm$ 0.003 $\star$	0.936 $\pm$ 0.002
	credit-a	0.693 $\pm$ 0.019 $\odot$	0.655 $\pm$ 0.031	0.604 $\pm$ 0.014 $\star$	0.686 $\pm$ 0.029$\odot$	0.601 $\pm$ 0.015 $\star$	0.634 $\pm$ 0.014 $\star$	0.642 $\pm$ 0.019
	dna	0.664 $\pm$ 0.017 $\star$	0.618 $\pm$ 0.030 $\star$	0.645 $\pm$ 0.017 $\star$	0.654 $\pm$ 0.047 $\star$	0.649 $\pm$ 0.016 $\star$	0.612 $\pm$ 0.012 $\star$	0.683 $\pm$ 0.033
	ijcnn1	0.173 $\pm$ 0.001 $\odot$	0.108 $\pm$ 0.012 $\star$	0.172 $\pm$ 0.006 $\odot$	0.108 $\pm$ 0.012 $\star$	0.172 $\pm$ 0.006 $\odot$	0.277 $\pm$ 0.006$\odot$	0.141 $\pm$ 0.009
	magic04	0.787 $\pm$ 0.000 $\odot$	0.798 $\pm$ 0.001$\odot$	0.673 $\pm$ 0.002 $\star$	0.798 $\pm$ 0.001$\odot$	0.673 $\pm$ 0.002 $\star$	0.718 $\pm$ 0.003 $\star$	0.760 $\pm$ 0.002
	mushroom	0.616 $\pm$ 0.019 $\star$	0.728 $\pm$ 0.006 $\star$	0.678 $\pm$ 0.007 $\star$	0.727 $\pm$ 0.005 $\star$	0.678 $\pm$ 0.007 $\star$	0.591 $\pm$ 0.006 $\star$	0.815 $\pm$ 0.003
	musk2	0.259 $\pm$ 0.003 $\star$	0.221 $\pm$ 0.010 $\star$	0.432 $\pm$ 0.008 $\star$	0.344 $\pm$ 0.009 $\star$	0.458 $\pm$ 0.008 $\star$	0.360 $\pm$ 0.012 $\star$	0.552 $\pm$ 0.013
	phishing	0.713 $\pm$ 0.003 $\star$	0.832 $\pm$ 0.003 $\star$	0.790 $\pm$ 0.003 $\star$	0.834 $\pm$ 0.003 $\star$	0.794 $\pm$ 0.003 $\star$	0.654 $\pm$ 0.003 $\star$	0.844 $\pm$ 0.003
	rna	0.470 $\pm$ 0.001 $\star$	0.591 $\pm$ 0.004 $\star$	0.577 $\pm$ 0.002 $\star$	0.639 $\pm$ 0.004 $\star$	0.581 $\pm$ 0.002 $\star$	0.487 $\pm$ 0.002 $\star$	0.676 $\pm$ 0.003
	sensors	0.640 $\pm$ 0.023 $\star$	0.901 $\pm$ 0.002 $\star$	0.899 $\pm$ 0.002 $\star$	0.903 $\pm$ 0.002 $\star$	0.899 $\pm$ 0.003 $\star$	0.601 $\pm$ 0.004 $\star$	0.906 $\pm$ 0.003
	spambase	0.528 $\pm$ 0.004 $\star$	0.659 $\pm$ 0.011 $\star$	0.643 $\pm$ 0.007 $\star$	0.681 $\pm$ 0.008$\star$	0.645 $\pm$ 0.007 $\star$	0.532 $\pm$ 0.009 $\star$	0.736 $\pm$ 0.006
	splice	0.668 $\pm$ 0.022$\odot$	0.577 $\pm$ 0.026 $\star$	0.554 $\pm$ 0.006 $\star$	0.579 $\pm$ 0.027 $\star$	0.556 $\pm$ 0.007 $\star$	0.618 $\pm$ 0.005 $\odot$	0.613 $\pm$ 0.010
	svmguide1	0.715 $\pm$ 0.015 $\star$	0.779 $\pm$ 0.007 $\star$	0.712 $\pm$ 0.005 $\star$	0.777 $\pm$ 0.007 $\star$	0.713 $\pm$ 0.004 $\star$	0.697 $\pm$ 0.005 $\star$	0.839 $\pm$ 0.003
	wdbc	0.516 $\pm$ 0.015 $\star$	0.179 $\pm$ 0.054 $\star$	0.559 $\pm$ 0.018 $\star$	0.317 $\pm$ 0.054 $\star$	0.621 $\pm$ 0.020 $\star$	0.511 $\pm$ 0.020 $\star$	0.670 $\pm$ 0.025
win-tie-lose		41-0-4	41-2-2	40-3-2	47-3-5	40-1-4	37-2-6	-

In the context of the Nemenyi test, (1) for the general evolution scenarios, critical difference is computed as 1.551, with parameters set at $q_{0.05} = 2.948$, $k = 9$, and $n = 60$; (2) for the synchronized evolution scenarios, the critical difference is computed as 1.1627, with parameters set at $q_{0.05} = 2.948$, $k = 7$, and $n = 60$; (3) for the overlap evolution scenarios, the critical difference is computed as 1.1627, with parameters set at $q_{0.05} = 2.948$, $k = 7$, and $n = 60$. As we can see from Table 5, OLEF-KL consistently demonstrates competitive performance within the top two positions relative to other competitors. This observation indicates that OLEF-KL possesses the capability to adapt not only to the previously strictly presumed evolving features but also to a broader spectrum of generally evolving features.

5.7. Parameter study

We proceed to examine the influence of key parameters within the OLEF-KL algorithm. The examination were conducted under general evolution scenario. Our algorithm includes three crucial parameters: the decay rate ρ , the coefficient η , and the bandwidth σ used in the Gaussian kernel function. We chose values for these parameters as follows: ρ from the set $\{0.9, 0.99, 0.999, 0.9999\}$, η from the set $\{0.1, 0.2, 0.5, 1, 2, 5, 10\}$, and σ from the set $\{0.1, 0.2, 0.3, 0.4, 0.5, 1, 2, 5\}$.

During the analysis of each parameter, we maintain the other parameters at constant values, specifically, $\rho = 0.999$, $\eta = 1$, and $\sigma = 0.2$. To offer a comprehensive parameter analysis, we employ normalized accuracy, where the highest accuracy is scaled to 1. This metric provides insight into performance variations with varying parameters, while mitigating the influence of dataset complexity. The results of our parameter study are shown in Fig. 3, leading to the following key observations.

- Parameter ρ : The observations from Fig. 3(a) reveal a positive correlation between the performance of OLEF-KL and the magnitude of the decay parameter ρ . As ρ is increases, the performance improves. Apparently, the larger ρ expanding the capacity of the model, enabling it to support a more complex model and thus aiding in capturing intricate patterns inherent in the data. However, it is crucial to note that selecting the value of $\rho = 1$ may come at the cost of reduced computational efficiency. Hence, a selection of the decay parameter is important to keep a balance between optimizing performance and maintaining computational efficiency.
- Parameter η : The insights derived from Fig. 3(b) underscore the dataset-specific nature of the coefficient η . It is evident that neither excessively large nor exceedingly small values of η can ensure

Table 4

Performance comparison under the evolving with overlap scenarios (mean $\pm$ std). $\star/\odot$ indicate that OLEF-KL is significantly better/worse than competitors (hypothesis supported by paired t-tests at 0.05 significant level). The best results are indicated in **bold**.

Metric	Datasets	NOGD	NPA	FESL-c	FESL-c	PUFE	OLEF-KL
ACC	HTRU2	0.911 $\pm$ 0.001 $\star$	0.945 $\pm$ 0.005 $\star$	0.925 $\pm$ 0.004 $\star$	0.925 $\pm$ 0.005 $\star$	0.925 $\pm$ 0.005 $\star$	0.974 $\pm$ 0.001
	TUANDROMD	0.798 $\pm$ 0.000 $\star$	0.873 $\pm$ 0.012 $\star$	0.814 $\pm$ 0.009 $\star$	0.815 $\pm$ 0.009 $\star$	0.816 $\pm$ 0.009 $\star$	0.926 $\pm$ 0.009
	credit-a	0.627 $\pm$ 0.017 $\star$	0.699 $\pm$ 0.022 $\star$	0.672 $\pm$ 0.014 $\star$	0.677 $\pm$ 0.017 $\star$	0.678 $\pm$ 0.019 $\star$	0.826 $\pm$ 0.012
	dna	0.587 $\pm$ 0.036 $\star$	0.659 $\pm$ 0.013 $\star$	0.696 $\pm$ 0.026	0.689 $\pm$ 0.025	0.697 $\pm$ 0.028	0.690 $\pm$ 0.015
	ijcnn1	0.900 $\pm$ 0.001 $\star$	0.889 $\pm$ 0.002 $\star$	0.903 $\pm$ 0.001 $\star$	0.902 $\pm$ 0.001 $\star$	0.902 $\pm$ 0.001 $\star$	0.928 $\pm$ 0.004
	magic04	0.650 $\pm$ 0.001 $\star$	0.632 $\pm$ 0.005 $\star$	0.677 $\pm$ 0.006 $\star$	0.679 $\pm$ 0.006 $\star$	0.679 $\pm$ 0.006 $\star$	0.756 $\pm$ 0.006
	mushroom	0.787 $\pm$ 0.048 $\star$	0.881 $\pm$ 0.006 $\star$	0.850 $\pm$ 0.017 $\star$	0.850 $\pm$ 0.017 $\star$	0.852 $\pm$ 0.018 $\star$	0.992 $\pm$ 0.001
	musk2	0.598 $\pm$ 0.019 $\star$	0.601 $\pm$ 0.008 $\star$	0.622 $\pm$ 0.013 $\star$	0.616 $\pm$ 0.008 $\star$	0.618 $\pm$ 0.008 $\star$	0.896 $\pm$ 0.008
	phishing	0.603 $\pm$ 0.034 $\star$	0.744 $\pm$ 0.021 $\star$	0.728 $\pm$ 0.031 $\star$	0.731 $\pm$ 0.027 $\star$	0.727 $\pm$ 0.032 $\star$	0.821 $\pm$ 0.025
	rna	0.670 $\pm$ 0.009 $\star$	0.811 $\pm$ 0.009 $\star$	0.799 $\pm$ 0.007 $\star$	0.800 $\pm$ 0.007 $\star$	0.801 $\pm$ 0.007 $\star$	0.891 $\pm$ 0.002
	sensors	0.664 $\pm$ 0.006 $\star$	0.673 $\pm$ 0.004 $\star$	0.715 $\pm$ 0.045$\odot$	0.717 $\pm$ 0.036$\odot$	0.687 $\pm$ 0.030	0.676 $\pm$ 0.004
	spambase	0.737 $\pm$ 0.005 $\star$	0.792 $\pm$ 0.013 $\star$	0.756 $\pm$ 0.012 $\star$	0.752 $\pm$ 0.009 $\star$	0.753 $\pm$ 0.009 $\star$	0.870 $\pm$ 0.009
	splice	0.516 $\pm$ 0.008 $\star$	0.560 $\pm$ 0.011 $\star$	0.561 $\pm$ 0.009 $\star$	0.562 $\pm$ 0.010 $\star$	0.566 $\pm$ 0.010 $\star$	0.637 $\pm$ 0.007
	svmguide1	0.633 $\pm$ 0.010 $\star$	0.720 $\pm$ 0.004 $\star$	0.704 $\pm$ 0.038 $\star$	0.707 $\pm$ 0.039 $\star$	0.707 $\pm$ 0.040 $\star$	0.893 $\pm$ 0.002
	wdbc	0.615 $\pm$ 0.011 $\star$	0.648 $\pm$ 0.016 $\star$	0.625 $\pm$ 0.007 $\star$	0.623 $\pm$ 0.010 $\star$	0.622 $\pm$ 0.012 $\star$	0.828 $\pm$ 0.014
AUC	HTRU2	0.521 $\pm$ 0.009 $\star$	0.845 $\pm$ 0.010 $\star$	0.924 $\pm$ 0.013$\odot$	0.922 $\pm$ 0.015$\odot$	0.923 $\pm$ 0.017$\odot$	0.888 $\pm$ 0.005
	TUANDROMD	0.508 $\pm$ 0.005 $\star$	0.828 $\pm$ 0.014 $\star$	0.774 $\pm$ 0.038 $\star$	0.768 $\pm$ 0.040 $\star$	0.768 $\pm$ 0.041 $\star$	0.867 $\pm$ 0.013
	credit-a	0.646 $\pm$ 0.031 $\star$	0.692 $\pm$ 0.024 $\star$	0.744 $\pm$ 0.037 $\star$	0.734 $\pm$ 0.034 $\star$	0.744 $\pm$ 0.038 $\star$	0.803 $\pm$ 0.017
	dna	0.593 $\pm$ 0.046 $\star$	0.659 $\pm$ 0.017 $\star$	0.776 $\pm$ 0.027$\odot$	0.762 $\pm$ 0.026$\odot$	0.768 $\pm$ 0.026$\odot$	0.693 $\pm$ 0.017
	ijcnn1	0.505 $\pm$ 0.004 $\star$	0.681 $\pm$ 0.006$\odot$	0.683 $\pm$ 0.028	0.674 $\pm$ 0.027	0.675 $\pm$ 0.028	0.674 $\pm$ 0.013
	magic04	0.503 $\pm$ 0.003 $\star$	0.603 $\pm$ 0.005 $\star$	0.720 $\pm$ 0.025	0.713 $\pm$ 0.026	0.716 $\pm$ 0.025	0.716 $\pm$ 0.006
	mushroom	0.782 $\pm$ 0.044 $\star$	0.883 $\pm$ 0.006 $\star$	0.911 $\pm$ 0.009 $\star$	0.911 $\pm$ 0.009 $\star$	0.912 $\pm$ 0.009 $\star$	0.992 $\pm$ 0.001
	musk2	0.672 $\pm$ 0.012 $\star$	0.659 $\pm$ 0.011 $\star$	0.788 $\pm$ 0.015$\odot$	0.775 $\pm$ 0.008 $\odot$	0.779 $\pm$ 0.007 $\odot$	0.751 $\pm$ 0.016
	phishing	0.591 $\pm$ 0.066 $\star$	0.744 $\pm$ 0.021 $\star$	0.813 $\pm$ 0.026	0.808 $\pm$ 0.028	0.807 $\pm$ 0.029	0.820 $\pm$ 0.023
	rna	0.503 $\pm$ 0.003 $\star$	0.796 $\pm$ 0.009 $\star$	0.888 $\pm$ 0.004$\odot$	0.888 $\pm$ 0.004$\odot$	0.888 $\pm$ 0.004$\odot$	0.883 $\pm$ 0.002
	sensors	0.639 $\pm$ 0.079 $\star$	0.675 $\pm$ 0.004 $\star$	0.820 $\pm$ 0.039 $\odot$	0.828 $\pm$ 0.032$\odot$	0.799 $\pm$ 0.028 $\odot$	0.676 $\pm$ 0.004
	spambase	0.740 $\pm$ 0.005 $\star$	0.792 $\pm$ 0.014 $\star$	0.808 $\pm$ 0.016 $\star$	0.802 $\pm$ 0.012 $\star$	0.802 $\pm$ 0.012 $\star$	0.850 $\pm$ 0.009
	splice	0.511 $\pm$ 0.010 $\star$	0.560 $\pm$ 0.012 $\star$	0.587 $\pm$ 0.015 $\star$	0.586 $\pm$ 0.014 $\star$	0.594 $\pm$ 0.018 $\star$	0.642 $\pm$ 0.008
	svmguide1	0.641 $\pm$ 0.022 $\star$	0.707 $\pm$ 0.004 $\star$	0.839 $\pm$ 0.017 $\star$	0.838 $\pm$ 0.017 $\star$	0.841 $\pm$ 0.015 $\star$	0.891 $\pm$ 0.002
	wdbc	0.514 $\pm$ 0.011 $\star$	0.640 $\pm$ 0.020 $\star$	0.683 $\pm$ 0.043 $\star$	0.679 $\pm$ 0.040 $\star$	0.691 $\pm$ 0.049 $\star$	0.823 $\pm$ 0.017
F1	HTRU2	0.057 $\pm$ 0.021 $\star$	0.704 $\pm$ 0.026 $\star$	0.301 $\pm$ 0.063 $\star$	0.303 $\pm$ 0.069 $\star$	0.304 $\pm$ 0.072 $\star$	0.843 $\pm$ 0.004
	TUANDROMD	0.002 $\pm$ 0.003 $\star$	0.699 $\pm$ 0.028 $\star$	0.164 $\pm$ 0.080 $\star$	0.173 $\pm$ 0.079 $\star$	0.180 $\pm$ 0.079 $\star$	0.805 $\pm$ 0.026
	credit-a	0.394 $\pm$ 0.067 $\star$	0.670 $\pm$ 0.024 $\star$	0.527 $\pm$ 0.035 $\star$	0.551 $\pm$ 0.035 $\star$	0.542 $\pm$ 0.043 $\star$	0.805 $\pm$ 0.015
	dna	0.526 $\pm$ 0.122 $\star$	0.651 $\pm$ 0.013 $\star$	0.664 $\pm$ 0.054	0.659 $\pm$ 0.052 $\star$	0.670 $\pm$ 0.057	0.681 $\pm$ 0.015
	ijcnn1	0.025 $\pm$ 0.007 $\star$	0.419 $\pm$ 0.011 $\star$	0.022 $\pm$ 0.009 $\star$	0.026 $\pm$ 0.010 $\star$	0.027 $\pm$ 0.010 $\star$	0.465 $\pm$ 0.043
	magic04	0.016 $\pm$ 0.007 $\star$	0.480 $\pm$ 0.006 $\star$	0.181 $\pm$ 0.032 $\star$	0.189 $\pm$ 0.033 $\star$	0.190 $\pm$ 0.034 $\star$	0.615 $\pm$ 0.010
	mushroom	0.773 $\pm$ 0.052 $\star$	0.876 $\pm$ 0.007 $\star$	0.840 $\pm$ 0.016 $\star$	0.840 $\pm$ 0.016 $\star$	0.842 $\pm$ 0.017 $\star$	0.992 $\pm$ 0.001
	musk2	0.377 $\pm$ 0.007 $\star$	0.364 $\pm$ 0.009 $\star$	0.402 $\pm$ 0.011 $\star$	0.394 $\pm$ 0.005 $\star$	0.398 $\pm$ 0.004 $\star$	0.590 $\pm$ 0.036
	phishing	0.198 $\pm$ 0.129 $\star$	0.710 $\pm$ 0.024 $\star$	0.618 $\pm$ 0.081 $\star$	0.632 $\pm$ 0.058 $\star$	0.614 $\pm$ 0.087 $\star$	0.792 $\pm$ 0.030
	rna	0.024 $\pm$ 0.051 $\star$	0.713 $\pm$ 0.015 $\star$	0.617 $\pm$ 0.027 $\star$	0.620 $\pm$ 0.027 $\star$	0.621 $\pm$ 0.027 $\star$	0.837 $\pm$ 0.002
	sensors	0.670 $\pm$ 0.086 $\star$	0.670 $\pm$ 0.003 $\star$	0.715 $\pm$ 0.051$\odot$	0.715 $\pm$ 0.048$\odot$	0.692 $\pm$ 0.047	0.678 $\pm$ 0.005
	spambase	0.666 $\pm$ 0.007 $\star$	0.752 $\pm$ 0.015 $\star$	0.693 $\pm$ 0.017 $\star$	0.689 $\pm$ 0.014 $\star$	0.689 $\pm$ 0.014 $\star$	0.827 $\pm$ 0.012
	splice	0.267 $\pm$ 0.160 $\star$	0.543 $\pm$ 0.011 $\star$	0.399 $\pm$ 0.048 $\star$	0.407 $\pm$ 0.043 $\star$	0.418 $\pm$ 0.046 $\star$	0.624 $\pm$ 0.008
	svmguide1	0.321 $\pm$ 0.049 $\star$	0.676 $\pm$ 0.005 $\star$	0.507 $\pm$ 0.095 $\star$	0.514 $\pm$ 0.098 $\star$	0.516 $\pm$ 0.099 $\star$	0.876 $\pm$ 0.002
	wdbc	0.095 $\pm$ 0.088 $\star$	0.529 $\pm$ 0.023 $\star$	0.055 $\pm$ 0.055 $\star$	0.067 $\pm$ 0.063 $\star$	0.070 $\pm$ 0.068 $\star$	0.757 $\pm$ 0.022
win-tie-lose		44-1-0	43-1-1	33-5-7	34-4-7	33-7-5	-

Table 5

Summarization of rankings under difference evolution scenarios, in term of ACC, AUC and F1 Score. The top-2 results are indicated in **bold**.

Sc.	Metric	NOGD	NPA	OLSF _{OGD}	OLSF	OLVF	OLCF	OLI2DS	RSOL	FF	OPID	FESL-s	FESL-c	PUFE	OLEF-KL	CD
Genl	ACC	7.017	8.200	3.783	5.883	3.033	4.933	5.483	4.767	-	-	-	-	-	1.900	1.551
	AUC	8.067	7.617	4.800	5.350	3.500	5.917	3.083	3.767	-	-	-	-	-	2.900	
	F1	7.367	8.000	4.050	5.517	3.200	5.383	4.983	4.300	-	-	-	-	-	2.200	
Sync	ACC	3.742	4.758	2.925	4.375	-	-	-	-	6.817	3.850	-	-	-	1.533	1.162
	AUC	4.508	4.092	3.692	3.542	-	-	-	-	6.983	3.333	-	-	-	1.850	
	F1	3.975	4.725	3.425	4.392	-	-	-	-	5.533	4.083	-	-	-	1.867	
Lap	ACC	5.867	3.333	-	-	-	-	-	-	-	-	3.600	3.733	3.133	1.333	1.947
	AUC	5.933	4.667	-	-	-	-	-	-	-	-	3.600	2.000	2.467	2.333	
	F1	5.667	2.667	-	-	-	-	-	-	-	-	3.867	4.333	3.267	1.200	

- Genl: General Evolution; Sync: Synchronized Evolution; Lap: Overlap Evolution.

- CD: Critical Difference. $CD = q_{0.05} \sqrt{k(k+1)/6n}$, where k is the number of algorithms and n is the number of datasets.

$q_{0.05}$ is the critical range value at the significance level of 0.05.

a satisfactory model performance. Empirical evidence suggests that setting η to a value of 0.1 serves as a practical guideline to attain a satisfying performance level across datasets.

- Parameter σ : The findings presented in Fig. 3(c) emphasize that the utilization of excessively large values for the parameter σ within the Gaussian kernel results in unacceptable performance.

This outcome can be principally attributed to the enlargement of σ , which leads to a wide and flat Gaussian shape. Within this context, the kernel allocates nearly uniform weights to all data points across the feature space. Consequently, the kernel smooths out variations in the data, making it challenging for the model to capture fine-grained patterns in the data.

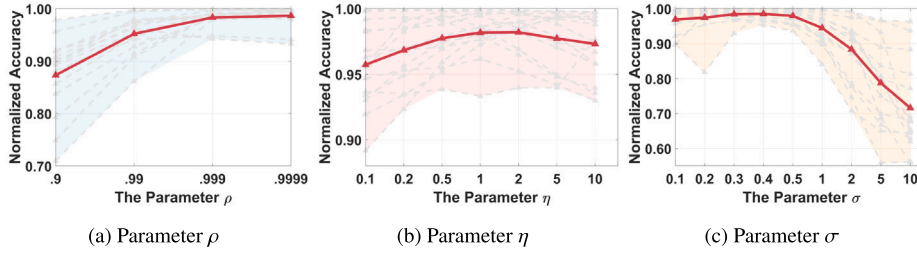


Fig. 3. Parameter study on parameter (a) ρ , (b) η and (c) σ . Each dashed gray curve denotes performance variation on a dataset with varying parameter. The red line denotes the averaged performance on all datasets. The results are presented in terms of normalized accuracy, with the highest accuracy scaled to 1 for a fair comparison. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

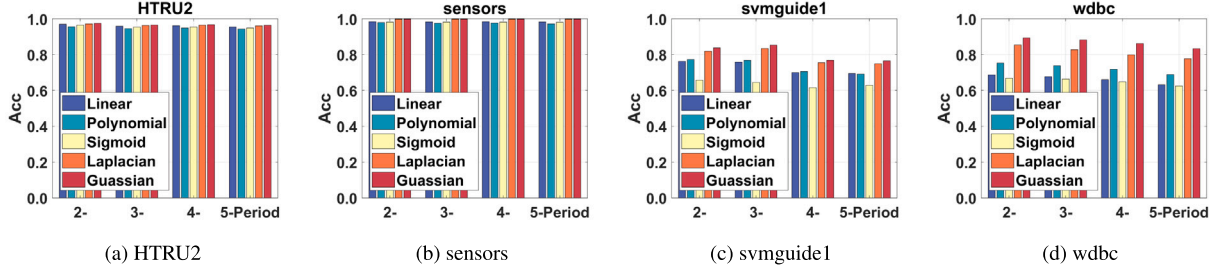


Fig. 4. Performance comparison when OLEF-KL using different kernel functions across four datasets, under 2-period general evolving scenarios..

Table 6

Time comparison of different methods under 2-period synchronized evolution.

Time(s)	NOGD	OPID	NPA	OLVF	OLI2DS	OLEF _{0.9}	OLEF ₁
HTRU2	0.056	8.468	0.068	0.236	0.415	0.484	0.649
TUANDROMD	0.033	6.415	0.033	0.144	0.186	0.410	1.407
credit-a	0.003	0.315	0.003	0.011	0.017	0.025	0.023
dna	0.028	1.374	0.024	0.046	0.068	0.272	0.465
ijcnn1	0.171	24.541	0.184	0.736	1.304	1.620	13.281
magic04	0.078	9.209	0.084	0.312	0.495	0.587	2.953
mushroom	0.032	4.015	0.034	0.127	0.209	0.266	0.665
musk2	0.046	7.379	0.047	0.153	0.268	0.406	4.764
phishing	0.058	6.431	0.052	0.192	0.337	0.445	3.630
rna	0.231	28.391	0.229	0.843	1.407	1.679	11.844
sensors	0.041	5.550	0.039	0.165	0.272	0.398	1.463
spambase	0.025	2.536	0.024	0.083	0.138	0.181	0.848
splice	0.020	1.853	0.019	0.065	0.101	0.126	0.480
svmguide1	0.031	3.495	0.031	0.115	0.179	0.203	0.368
wdbc	0.003	0.284	0.003	0.010	0.016	0.018	0.018

5.8. Kernel study

This section conducts a comparative analysis of the performance of OLEF-KL by employing several distinct kernel functions: the linear, polynomial, sigmoid and Laplacian and the Gaussian kernel function.. The assessment of model performance was based on the metric of ACC, and the outcomes are presented in Fig. 4.

The selection of the kernel function depends on the characteristics of the data. For instance, in the case of the *HTRU2* and *sensors* datasets, all kernel functions yield comparable results. However, in the *svmguide1* dataset, the sigmoid kernel performs less effectively than simple linear learning. These observations indicate that an inappropriate kernel function or its parameters may even degrade performance.

In most cases, OLEF-KL with the Gaussian kernel function outperforms its variants using other kernel functions. Due to the Gaussian kernel's inherent simplicity, we recommend its adoption in OLEF-KL implementations.

5.9. Time analysis under evolving scenarios

Table 6 presents the computation time (in seconds) for various algorithms in the synchronized evolving feature data streams. A notable

observation is that OLEF-KL ($\rho = 1$) generally exhibits longer execution times compared to many of the other algorithms across a wide range of datasets. This suggests that OLEF-KL may be computationally more demanding. As theoretically analyzed in Table S9 in *supplementary material*, the computational of OLEF-KL is $\mathcal{O}(t|\mathcal{d}_t|)$ which is linear with the time. A crucial technique for reducing the computational demands of OLEF-KL is the incorporation of a decay parameter $\rho < 1$, and this results in the computational complexity of $\mathcal{O}\left(\frac{\ln \epsilon}{\ln \rho} |\mathcal{d}_t| t\right)$, where $\epsilon > 0$ is the truncation parameter and $\rho < 1$ is the decaying parameter. Particularly, the term $\frac{\ln \epsilon}{\ln \rho}$ represents a constant factor determined by these fixed hyperparameters. This analysis demonstrates that the computational cost of OLEF-KL for new feature learning scales linearly with the feature dimension $|\mathcal{d}_t|$, ensuring efficient adaptation even as the feature space grows.

6. Conclusion

This study addresses a general scenario of evolving features that refrains from making interaction assumptions or imposing evolution prerequisites, as shown in Fig. 1(e). Subsequently, the OLEF-KL algorithm is introduced. In particular,

- It can continuously adapt not only to the previously strictly assumed evolving features but also to generally evolving features.
- Theoretical analysis shows a sublinear regret ($\mathcal{O}(\sqrt{T})$) of OLEF-KL, highlighting its theoretical effectiveness of learning within dynamic feature space.
- Experiments on both synthetic data and public datasets demonstrate the competitive performance of OLEF-KL in various feature evolution scenarios.

Future research could focus on modeling the relationships between features when faced with more complex evolving characteristics. For example, practical features might undergo replacement rather than a simple disappearance. In such instances, forgetting replaced features in OLEF-KL could lead to the loss of important information. Hence, exploring the interplay between new and old features could enhance the performance of online learning under feature evolution, particularly when there is no strict assumption about the interaction patterns of evolving features.

CRedit authorship contribution statement

Han Zhou: Writing – review & editing, Writing – original draft, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Hongpeng Yin:** Writing – review & editing, Software, Methodology, Investigation, Funding acquisition, Conceptualization. **Bin Wang:** Writing – review & editing, Validation, Methodology, Investigation, Conceptualization. **Chenglin Liao:** Writing – review & editing, Visualization, Methodology, Investigation, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Hongpeng Yin reports administrative support, article publishing charges, and equipment, drugs, or supplies were provided by National Natural Science Foundation of China. Hongpeng Yin reports a relationship with National Natural Science Foundation of China that includes: funding grants. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 62273062, in part by the National Key R&D Program of China under Grant 2023YFB3308800, in part by the Fundamental Research Funds for the Central Universities under Grant 2023CDJKYJH029, in part by the Major Key Project of PCL under Grant PCL2024A05, in part by the Natural Science Foundation of Chongqing under Grant CSTB2023YSZX-JSX00054.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.patcog.2025.111719>.

Data availability

Data will be made available on request.

References

- [1] Elad Hazan, et al., Introduction to online convex optimization, *Found. Trends® Optim.* 2 (3–4) (2016) 157–325.
- [2] Łukasz Korycki, Bartosz Krawczyk, Instance exploitation for learning temporary concepts from sparsely labeled drifting data streams, *Pattern Recognit.* 129 (2022) 108749.
- [3] Nuwan Gunasekara, Bernhard Pfahringer, Heitor Murilo Gomes, Albert Bifet, Survey on online streaming continual learning, in: *International Joint Conference on Artificial Intelligence*, 2023, pp. 6628–6637.
- [4] Han Zhou, Shin Matsushima, Online learning under capricious feature data streams, in: *Proceedings of the Annual Conference of JSAI, JSAI2023*, 2023, 2D4GS201–2D4GS201.
- [5] Han Zhou, Hongpeng Yin, Yan Qin, Chau Yuen, Industrial fault diagnosis with incremental learning capability under varying sensory data, *IEEE Trans. Syst. Man Cybern.: Syst.* 55 (2) (2025) 1322–1333.
- [6] Qin Zhang, Peng Zhang, Guodong Long, Wei Ding, Chengqi Zhang, Xindong Wu, Towards mining trapezoidal data streams, in: *IEEE International Conference on Data Mining*, 2015, pp. 1111–1116.
- [7] Qin Zhang, Peng Zhang, Guodong Long, Wei Ding, Chengqi Zhang, Xindong Wu, Online learning from trapezoidal data streams, *IEEE Trans. Knowl. Data Eng.* 28 (10) (2016) 2709–2723.
- [8] Bo-Jian Hou, Lijun Zhang, Zhi-Hua Zhou, Learning with feature evolvable streams, in: *Advances in Neural Information Processing Systems*, 2017, pp. 1416–1426.
- [9] Bo-Jian Hou, Lijun Zhang, Zhi-Hua Zhou, Prediction with unpredictable feature evolution, *IEEE Trans. Neural Netw. Learn. Syst.* 33 (10) (2021) 5706–5715.
- [10] Chenping Hou, Zhi-Hua Zhou, One-pass learning with incremental and decremental features, *IEEE Trans. Pattern Anal. Mach. Intell.* 40 (11) (2017) 2776–2792.
- [11] Martin Zinkevich, Online convex programming and generalized infinitesimal gradient ascent, in: *International Conference on Machine Learning*, 2003, pp. 928–936.
- [12] Han Zhou, Hongpeng Yin, Xuanhong Deng, Yuyu Huang, Online harmonizing gradient descent for imbalanced data streams one-pass classification, in: *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, 2023, pp. 2468–2475.
- [13] Koby Crammer, Ofer Dekel, Joseph Keshet, Shai Shalev-Shwartz, Yoram Singer, Online passive-aggressive algorithms, *J. Mach. Learn. Res.* 7 (19) (2006) 551–585.
- [14] Han Zhou, Hongpeng Yin, Dandan Zhao, Li Cai, Incremental learning and conditional drift adaptation for nonstationary industrial process fault diagnosis, *IEEE Trans. Ind. Inform.* 19 (4) (2023) 5935–5944.
- [15] Haipeng Luo, Alekh Agarwal, Nicolo Cesa-Bianchi, John Langford, Efficient second order online learning by sketching, *Adv. Neural Inf. Process. Syst.* 29 (2016).
- [16] Mark Dredze, Koby Crammer, Fernando Pereira, Confidence-weighted linear classification, in: *International Conference on Machine Learning*, 2008, pp. 264–271.
- [17] Cui Yu, Rui Zhang, Yaochun Huang, Hui Xiong, High-dimensional knn joins with incremental updates, *Geoinformatica* 14 (2010) 55–82.
- [18] Yingying Chen, Xiaowei Yang, Online adaptive kernel learning with random features for large-scale nonlinear classification, *Pattern Recognit.* 131 (2022) 108862.
- [19] Abrar Alam, Akshay Malhotra, Ioannis D. Schizas, Online kernel-based clustering, *Pattern Recognit.* 158 (2025) 111009.
- [20] Ofer Dekel, Shai Shalev-Shwartz, Yoram Singer, The forgetron: A kernel-based perceptron on a budget, *SIAM J. Comput.* 37 (5) (2008) 1342–1372.
- [21] Peilin Zhao, Jialei Wang, Pengcheng Wu, Rong Jin, Steven CH Hoi, Fast bounded online gradient descent algorithms for scalable kernel-based online learning, in: *International Conference on Machine Learning*, 2012, pp. 1075–1082.
- [22] Francesco Orabona, Joseph Keshet, Barbara Caputo, Bounded kernel-based online learning, *J. Mach. Learn. Res.* 10 (11) (2009).
- [23] Shilin Gu, Yuhua Qian, Chenping Hou, Learning with incremental instances and features, *IEEE Trans. Neural Netw. Learn. Syst.* (2023) 1–15.
- [24] Yanfang Liu, Xiaocong Fan, Wenbin Li, Yang Gao, Online passive-aggressive active learning for trapezoidal data streams, *IEEE Trans. Neural Netw. Learn. Syst.* 34 (10) (2023) 6725–6739.
- [25] Ege Beyazit, Matin Hosseini, Anthony Maida, Xindong Wu, Learning simplified decision boundaries from trapezoidal data streams, in: *International Conference on Artificial Neural Networks*, 2018, pp. 508–517.
- [26] Di Wu, Yi He, Xin Luo, MengChu Zhou, A latent factor analysis-based approach to online sparse streaming feature selection, *IEEE Trans. Syst. Man Cybern.: Syst.* 52 (11) (2021) 6744–6758.
- [27] Chenping Hou, Ling-Li Zeng, Dewen Hu, Safe classification with augmented features, *IEEE Trans. Pattern Anal. Mach. Intell.* 41 (9) (2018) 2176–2192.
- [28] Haotian Ni, Shilin Gu, Ruidong Fan, Chenping Hou, Feature incremental learning with causality, *Pattern Recognit.* 146 (2024) 110033.

- [29] Yi He, Baijun Wu, Di Wu, Ege Beyazit, Sheng Chen, Xindong Wu, Toward mining capricious data streams: A generative approach, *IEEE Trans. Neural Netw. Learn. Syst.* 32 (3) (2020) 1228–1240.
- [30] Di Wu, Shengda Zhuo, Yu Wang, Zhong Chen, Yi He, Online semi-supervised learning with mix-typed streaming features, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, 2023, pp. 4720–4728.
- [31] Heng Lian, Yu Huang, Xingquan Zhu, Yi He, Utilitarian online learning from open-world soft sensing, in: *2024 IEEE International Conference on Data Mining, ICDM, 2024*, pp. 241–250.
- [32] Bo-Jian Hou, Yu-Hu Yan, Peng Zhao, Zhi-Hua Zhou, Storage fit learning with feature evolvable streams, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 7729–7736.
- [33] Zhen-Yu Zhang, Peng Zhao, Yuan Jiang, Zhi-Hua Zhou, Learning with feature and distribution evolvable streams, in: *International Conference on Machine Learning, 2020*, pp. 11317–11327.
- [34] Heng Lian, Di Wu, Bo-Jian Hou, Jian Wu, Yi He, Online learning from evolving feature spaces with deep variational models, *IEEE Trans. Knowl. Data Eng.* 36 (8) (2024) 4144–4162.
- [35] Sheng-Da Zhuo, Jin-Jie Qiu, Chang-Dong Wang, Shu-Qiang Huang, Online feature selection with varying feature spaces, *IEEE Trans. Knowl. Data Eng.* 36 (9) (2024) 4806–4819.
- [36] Han-Jia Ye, De-Chuan Zhan, Yuan Jiang, Zhi-Hua Zhou, Rectify heterogeneous models with semantic mapping, in: *International Conference on Machine Learning, 2018*, pp. 5630–5639.
- [37] Shilin Gu, Chao Xu, Dewen Hu, Chenping Hou, Adaptive learning for dynamic features and noisy labels, *IEEE Trans. Pattern Anal. Mach. Intell.* 47 (2) (2025) 1219–1237.
- [38] Ege Beyazit, Jeevithan Alagurajah, Xindong Wu, Online learning from data streams with varying feature spaces, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 2019, pp. 3232–3239.
- [39] Dianlong You, Jiawei Xiao, Yang Wang, Huigui Yan, Di Wu, Zhen Chen, Limin Shen, Xindong Wu, Online learning from incomplete and imbalanced data streams, *IEEE Trans. Knowl. Data Eng.* 35 (10) (2023) 10650–10665.
- [40] Dianlong You, Huigui Yan, Jiawei Xiao, Zhen Chen, Di Wu, Limin Shen, Xindong Wu, Online learning for data streams with incomplete features and labels, *IEEE Trans. Knowl. Data Eng.* 36 (9) (2024) 4820–4834.
- [41] Zhong Chen, Yi He, Di Wu, Huixin Zhan, Victor Sheng, Kun Zhang, Robust sparse online learning for data streams with streaming features, in: *Proceedings of the 2024 SIAM International Conference on Data Mining, SDM, SIAM, 2024*, pp. 181–189.
- [42] Jie Zhao, Yun Ling, Faliang Huang, Jiahai Wang, Eric W.K. See-To, Incremental feature selection for dynamic incomplete data using sub-tolerance relations, *Pattern Recognit.* 148 (2024) 110125.
- [43] Weihua Xu, Weirui Ye, Incremental feature selection: Parallel approach with local neighborhood rough sets and composite entropy, *Pattern Recognit.* 159 (2025) 111141.
- [44] Bernhard Schölkopf, Ralf Herbrich, Alex J. Smola, A generalized representer theorem, in: *International Conference on Computational Learning Theory, Springer, 2001*, pp. 416–426.